Debreceni SZC Mechwart András Gépipari és Informatikai Technikum

Szoftverfejlesztő és -tesztelő

Vizsgaremek

QuestBook

Tabletop RPG és társasjáték közösségi platform

Készítette: **Barna István 13F**
Szakács Máté 13F

2026.

Tartalomjegyzék

Tartalomjegyzék **2**

1. Bevezetés **5**

1.1 Röviden a projektről 5

1.2 Témaválasztás indoklása 6

1.3 Szerepjátékról röviden 6

1.4 Célkitűzések 7

1.5 Kinek szánjuk a weboldalt 7

2. Fejlesztői dokumentáció **8**

2.1 Bevezető 8

2.2 Feljesztői környezet hardverének ismertetése 9

2.3 Fejlesztés során használt szoftverek 10

2.4 Backend ismertetése 11

2.4.1 Backendhez használt technológiák 11

2.4.2 Adatbázis szerkezete 14

2.4.2.1. USER TÁBLA (UserData) 14

2.4.2.2. TOKENS TÁBLA (Token) 15

2.4.2.3. LOBBIES TÁBLA (Lobby) 16

2.4.2.4. LOBBYCONS TÁBLA (LobbyCon) 17

2.4.2.5. LOCATIONS TÁBLA (Location) 17

2.4.2.6. KOMMENTS TÁBLA (Komment) 18

2.4.2.7. REPS TÁBLA (Reputation) 18

2.4.2.8. BANNEDUSERS TÁBLA (BannedUser) 19

2.4.3 Api lekérések 20

2.4.3.1. Autentikációs Endpointok 20

2.4.3.2. Felhasználó Endpointok 20

2.4.3.4. Helyszín Endpointok 20

2.4.3.5. Lobby Endpointok 20

2.4.3.6. Közösségi Endpointok (Rep és Komment) 21

2.4.3.7. Admin Endpointok 21
2.4.3.8. Egyéb Endpointok 21
2.5 Tipikus algoritmusok bemutatása 22
2.5.1 Regisztrációs algoritmus 22
2.5.2 JWT Token generálás 23
2.5.3 Lobby létrehozása időpont-ütköző ellenőrzéssel 23
2.5.4 Játékos csatlakozása lobbyhoz 24
2.6 Tesztdokumentáció 25
2.7 Frontend ismertetése 29
2.7.1 Frontendhez használt technológiák 29
2.7.2 Állapotkezelés 30
2.7.3 Backend kommunikáció 31
2.7.4 Komponens struktúra 32
2.7.5 Felhasználói felület kialakítása 33
2.7.6 Fejlesztési nehézségek 33
2.8 Frontend fejlesztés során szerzett tapasztalatok 34
2.8.1 Hibakeresés 34
2.8.2 Kódszerkezet 34
2.8.3 Felhasználói élmény 35
2.8.4 Összegzés 35
2.9 További fejlesztési lehetőségek 35
3. Felhasználói dokumentáció 36
3.1 A program rövid bemutatása 36
3.2 Rendszerkövetelmények (hardver) 36
3.3 Szoftverkövetelmények 37
3.4 Telepítés és indítás 37
3.5 A program részletes használata 38
3.5.1 Főoldal – About us 38
3.5.2 Bejelentkezés: 39
3.5.3 Regisztráció 40
3.5.4 Bejelentkezett felhasználók funkciói 41
3.5.5 Helyszínek böngészése 42
3.5.6 Foglalás létrehozása (Reserve) 43

3.5.7 Csatlakozás lobbyhoz (Community nézet) ... 44
3.5.8 Kilépés lobbyból ... 45
3.5.9 Lobby törlése ... 45
3.5.10 Profil nézet ... 46
- Profil szerkesztése: ... 46

3.5.11 Játékos keresése ... 47
3.5.12 Kijelentkezés ... 48
3.5.13 Admin felület és lehetőségei ... 49
- Bejelentkezés az admin panelbe ... 49
- Elérhető admin műveletek ... 50

3.5.14 Hibajelzések összefoglalása ... 51
- Webes felület (frontend) ... 51
- Admin panel (WPF) ... 51

3.5.15 Elérhetőség ... 51
4. Csapatmunka megvalósítása ... 52
5. Összegzés ... 53

1. Bevezetés

1.1 Röviden a projektről

A **QuestBook** egy modern webes és asztali alkalmazás, amelynek célja asztali társasjátékok és szerepjátékok (tabletop RPG) közösségi szervezésének megkönnyítése.

A csapatkereső és közösségszervező platform lehetőséget biztosít a felhasználók számára, hogy játékalkalmakat szervezzenek, csatlakozzanak meglévő eseményekhez, valamint megfelelő helyszíneket találjanak a játékok lebonyolításához.

Főbb funkciók

- ✓ **Helyszínböngészés**

  A felhasználók Magyarország területén elérhető, társasjátékokra alkalmas helyszínek között kereshetnek, részletes leírásokkal és információkkal.
- ✓ **Lobbik (játékalkalmak) kezelése**
  - Új játékalkalmak létrehozása
  - Csatlakozás meglévő eseményekhez
  - Résztvevők kezelése
- ✓ **Csapatkereső rendszer**

  Azok a felhasználók, akik nem rendelkeznek csapattal, könnyedén találhatnak játékostársakat.
- ✓ **Foglaláskezelés**

  A játékalkalmakhoz tartozó helyszínek lefoglalása és kezelése egy központi rendszeren keresztül történik.
- ✓ **Értékelési rendszer (rep rendszer)**

  A felhasználók értékelhetik egymást, ezzel növelve a közösség megbízhatóságát.
- ✓ **Kommentelési lehetőség**

  A játékokhoz és felhasználókhoz kapcsolódó visszajelzések megosztása.
- ✓ **Helyszín hozzáadás**

  A felhasználók saját helyszínt is felajánlhatnak, amelyet az adminisztrátorok ellenőriznek és rögzítenek a rendszerben.

1.2 Témaválasztás indoklása

A projekt témáját személyes tapasztalatok és valós igények hívták életre. Korábban baráti társasággal rendszeresen jártunk szerepjáték-alkalmakra, és ezek szervezése során gyakran szembesültünk azzal, hogy nehéz megfelelő helyszínt találni, új játékosokat bevonni, vagy átlátható módon kezelni a csapatok szerveződését. Ezek a tapasztalatok rávilágítottak arra, hogy a társasjátékos és szerepjátékos közösségek számára hiányzik egy olyan egységes, könnyen használható platform, amely támogatja a játékalkalmak létrehozását, a csapatok összeállítását és a helyszínek kezelését.

A témaválasztást az is indokolja, hogy Magyarországon egyre növekvő érdeklődés mutatkozik a társasjátékok és a tabletop RPG-k iránt, azonban a közösségszervezés jelenleg több, egymástól független csatornán (közösségi média csoportok, fórumok, személyes ismeretségek) zajlik. Ez sokszor átláthatatlan, időigényes és kevésbé hatékony. A QuestBook célja, hogy ezt a szétszórt rendszert egységesítse, és egy modern, felhasználóbarát felületen tegye lehetővé a játékosok összekapcsolását.

A választott téma tehát egyszerre épül személyes motivációra és valós közösségi igényre. A projekt gyakorlati haszonnal bír, hiszen egy olyan problémára ad megoldást, amely sok játékost érint, és hozzájárulhat egy aktív, megbízható és támogató közösség kialakulásához.

1.3 Szerepjátékról röviden

A szerepjáték (tabletop RPG) egy olyan társasjáték, ahol a játékosok kitalált karakterek bőrébe bújva közösen alkotnak történetet.

Egy játékvezető irányítja a világot és a történéseket, míg a játékosok döntéseikkel alakítják a cselekményt. A játék során a képzelet, a kommunikáció és az együttműködés kap fő szerepet.
A szerepjáték egy közös történetmesélésen alapuló játék, ahol a játékosok karaktereket alakítanak, és döntéseikkel formálják a történetet.
A szerepjáték fejleszti a kreativitást, kommunikációt, problémamegoldást és a csapatmunkát.
Alap eszközök: szabálykönyv, karakterlap, dobókocka, írószer és karakterek.
A játékmester/mesélő irányítja a történetet, szabályokat is ő mondja el a játékosoknak.

1.4 Célkitűzések

A QuestBook interaktív weboldal fő célja egy aktív és megbízható közösség létrehozása, ahol a társasjátékok és szerepjátékok iránt érdeklődők egyszerűen szervezhetnek találkozókat, és új emberekkel ismerkedhetnek meg. A platform törekvése, hogy összekösse a játékosokat, megkönnyítse a szervezést, és elősegítse egy aktív, támogató közösség kialakulását.

1.5 Kinek szánjuk a weboldalt

A QuestBook weboldal elsősorban azok számára készült, akik érdeklődnek a társasjátékok és szerepjátékok (tabletop RPG) iránt, és szeretnének közösségben játszani.

- ✓ **Társasjáték- és szerepjáték kedvelők**

Olyan felhasználók, akik rendszeresen játszanak, vagy most szeretnének bekapcsolódni a hobbiba.

- ✓ **Csapattal nem rendelkező játékosok**

Azok, akiknek nincs saját játékosközösségük, és új csapatot keresnek.

- ✓ **Játékszervezők (Game Masterek)**

Akik saját játékalkalmakat szeretnének létrehozni, játékosokat toborozni és eseményeket menedzselni.

2. Fejlesztői dokumentáció

2.1 Bevezető

Az alkalmazás klasszikus kliens-szerver architektúrát követ, ahol a frontend, az admin panel és a backend teljesen elkülönül egymástól. Mindhárom komponens a közös REST API-n keresztül kommunikál: a React-alapú webes frontend és a WPF-alapú asztali admin panel is HTTP kérésekkel éri el a backend végpontjait.

A backend Visual Studioban készült C# programnyelven, az ASP.NET Core Web API sablon felhasználásával. Az API dokumentálására és tesztelésére Scalar API referencia lett alkalmazva a hagyományos Swagger UI helyett. A felhasználók azonosítása és a tokenek kezelése ASP.NET Core Identity Framework használata nélkül, kézzel implementált JWT-alapú autentikációval lett megvalósítva – a jelszavak hash-eléséhez kizárólag az **ASP.NET Core Identity** csomagból a **PasswordHasher<T>** osztály került felhasználásra, a teljes Identity rendszer (felhasználókezelés, szerepkörök, adatbázis séma) nélkül.

Az adatbázis felépítésének megtervezése a weboldal fejlesztésével párhuzamosan haladt, a fejlesztés során folyamatosan alakítva a struktúráját – táblák elhagyásával, újak hozzáadásával és meglévők módosításával. Ezt a Code-First megközelítés és az Entity Framework Core migrációs rendszere tette lehetővé, mivel az adatbázis séma a C# modellekből generálódik, és minden változás verziózott migrációként rögzül.

A **frontend** Visual Studio Code-ban készült TypeScript nyelven, React 19 keretrendszer és Vite fejlesztői eszköz felhasználásával. A React funkcionális komponensekre épül, **useState** és **useEffect** hook-okkal kezeli az alkalmazás állapotát és az aszinkron adatbetöltést. Az alkalmazás egyoldalas (SPA – Single Page Application) architektúrát követ: egyetlen App komponens vezérli a nézetváltásokat belső állapotkezeléssel, külső routing könyvtár (pl. React Router) használata nélkül. A backend API-val való kommunikáció egy központi **apiFetch** segédfüggvényen keresztül történik, amely a JWT tokent automatikusan csatolja minden kéréshez. A frontend és a backend közötti adatformátum-különbségeket mapper függvények hidalják át, amelyek az API válaszokat a felhasználói felület által várt típusokra alakítják.

Az **admin panel** egy önálló WPF (Windows Presentation Foundation) asztali alkalmazás, amely szintén Visual Studioban készült C# nyelven, .NET 10 keretrendszerre építve. Az admin panel kizárólag adminisztrátor jogosultsággal rendelkező felhasználók számára érhető el – az alkalmazás indításkor egy bejelentkezési ablakot (LoginWindow) jelenít meg, és csak sikeres admin autentikáció után nyitja meg a fő kezelőfelületet (MainWindow). Az admin panel **HttpClient**-tel kommunikál a backend REST API-val, a JWT tokent Bearer autentikációként csatolva minden kéréshez. A felületen **DataGrid** komponensek jelenítik meg a felhasználók, lobbyk és helyszínek adatait, a JSON válaszok deszerializálását pedig a Newtonsoft.Json könyvtár végzi. Az admin panel lehetővé teszi a felhasználók kezelését (szerepkör módosítás, figyelmeztetés, felfüggesztés, kitiltás), a lobbyk és helyszínek törlését, játékosok eltávolítását lobbyból, valamint új helyszínek hozzáadását.

2.2 Feljesztői környezet hardverének ismertetése

Minimum

- Operációs rendszer: Windows 10 (x64) – a WPF admin panel miatt kizárólag
- Processzor: 1.6 GHz-es, 64 bites (x64), legalább 2 mag
- RAM: 4 GB
- Tárhely: 2 GB szabad lemezterület – a .NET 10 SDK (~800 MB), a Node.js + npm csomagok (node_modules ~300 MB), az alkalmazás forráskódja (~50
- MB) és az SQLite adatbázis (~10 MB) együttesen
- Képernyőfelbontás: 1366x768 pixel
- Hálózat: a 7231-es (backend HTTPS) és az 5173-as (frontend HTTP) port

Ajánlott

- Operációs rendszer: Windows 11 (x64)
- Processzor: Intel i5 (8. generáció+) vagy egyenértékű, 4 mag
- RAM: 8 GB – kényelmes fejlesztés és tesztelés Visual Studio 2022-vel együtt, amely önmagában 2-4 GB memóriát igényelhet
- Tárhely: 5 GB szabad lemezterület SSD meghajtón – a Visual Studio 2022, a
- .NET SDK, a Node.js, a forráskód és az adatbázis együttes igénye, plusz a build artifactok (bin/obj mappák)
- Képernyőfelbontás: 1920x1080 pixel (Full HD)

2.3 Fejlesztés során használt szoftverek

A **Windows 11** modern és biztonságos operációs rendszer, amely stabil környezetet biztosít a fejlesztéshez és a projektben használt programok futtatásához.

A **Google Chrome** gyors és megbízható böngésző, amelyet a webes felület tesztelésére és a fejlesztéshez kapcsolódó online források elérésére használtam.

A **Microsoft Edge** böngészőt a webalkalmazás működésének ellenőrzésére és a frontend tesztelésre vettem igénybe.

A **Microsoft Word** szövegszerkesztő programot a dokumentáció elkészítéséhez, formázásához és rendszerezéséhez használtam.

A **GitHub** verziókezelő és projektkezelő platform, amely lehetővé teszi a kód tárolását, követését és a fejlesztési folyamat átlátható kezelését, így a forráskód verziókezeléséhez használtuk.

A **Visual Studio** integrált fejlesztőkörnyezet, amelyben a C# alapú alkalmazás fejlesztése, tesztelése és hibakeresése történt.
API dokumentációját és tesztelését pedig a Scalar API felülete támogatta.

A **Visual Studio Code** könnyű, bővíthető kódszerkesztő, amelyet kisebb módosításokhoz, fájlkezeléshez és gyors kódszerkesztéshez használtam.

2.4 Backend ismertetése

2.4.1 Backendhez használt technológiák

1. ASP.NET Core 10 (.NET 10)

A projekt a Microsoft legújabb webes keretrendszerét használja, amely a backendSzM.csproj fájlban net10.0 target framework-ként van megadva. Az ASP.NET Core egy cross-platform, nagy teljesítményű keretrendszer REST API-k építéséhez.

Miért ez lett választva:

- Beépített támogatás a Controller-alapú API fejlesztéshez (**AddControllers()**)
- Natív middleware pipeline (autentikáció, authorizáció, CORS, statikus fájlok) – a Program.cs-ben konfigurálva
- Kiváló teljesítmény és aktív Microsoft-támogatás
- Beépített Dependency Injection rendszer, amelyet a projekt a **DbContext** és egyéb szolgáltatások regisztrálásához használ

2. Entity Framework Core 10

Az ORM (Object-Relational Mapping) réteg, amely az adatbázis-kezelést végzi. A Microsoft.EntityFrameworkCore csomag biztosítja a modellek (pl. UserData, Lobby, Komment) és az adatbázis közötti leképezést.

Miért ez lett választva:

- Code-First megközelítés – a C# modellek alapján automatikusan generálódik az adatbázis séma
- Migrációk támogatása (**Microsoft.EntityFrameworkCore.Tools**) – az adatbázis-szerkezet verziózottan fejleszthető
- LINQ-alapú lekérdezések – a controllerekben használt **.Where()**, **.Select()**, **.FirstOrDefaultAsync()** hívások típusbiztos, olvasható lekérdezéseket tesznek lehetővé
- Kapcsolatok konfigurálása Fluent API-val – az OnModelCreating-ben definiált törlési viselkedések

3. SQLite

Könnyűsúlyú, fájl-alapú relációs adatbázis, amelyet a Program.cs-ben a **UseSqlite()** hívással konfigurálnak.

Miért ez lett választva:

- Nem igényel külön adatbázis-szerver telepítést – ideális fejlesztési és vizsgaprojekt környezethez
- Egyetlen fájlban tárolja az összes adatot (DndDb.db), így egyszerűen hordozható
- Teljes SQL-támogatás (táblák, foreign key-ek, indexek) minimális beállítással
- Az Entity Framework Core natívan támogatja, így az átállás más adatbázisra később egyszerű

4. JWT Bearer autentikáció

A JSON Web Token alapú autentikációs rendszer, amely a felhasználók bejelentkezését és az API-végpontok védelmét biztosítja. A Microsoft.AspNetCore.Authentication.JwtBearer és System.IdentityModel.Tokens.Jwt csomagok együtt végzik a tokenek generálását, aláírását és validálását.

Miért ez lett választva:

- Állapotmentes (stateless) autentikáció – a token önmagában tartalmazza a felhasználó azonosítóját, nevét és szerepkörét (ClaimTypes.NameIdentifier, ClaimTypes.Name, ClaimTypes.Role)
- Ideális SPA frontend-hez – a React frontend a tokent a kérések Authorization fejlécében küldi
- Szerepkör-alapú jogosultságkezelés – az [Authorize(Roles = "Admin")] és [Authorize(Roles = "User,Admin")] attribútumokkal szabályozható, ki melyik végpontot érheti el
- HMAC-SHA512 aláírás – biztosítja a token integritását és hitelességét
- Refresh Token mechanizmus – a felhasználónak nem kell folyamatosan újra bejelentkeznie

5. PasswordHasher (ASP.NET Core Identity)

A jelszavak biztonságos hash-eléséhez a projekt a **PasswordHasher<T>** osztályt használja a regisztráció és bejelentkezés során.

Miért ez lett választva:

- Automatikusan salt-olja a jelszavakat – minden hash egyedi, még azonos jelszavak esetén is
- Beépített verifikáció (**VerifyHashedPassword**) – a bejelentkezésnél a hash és az eredeti jelszó összehasonlítása biztonságosan történik
- Iparági szabvány (PBKDF2) – a brute-force támadások ellen ellenálló algoritmus
- Az ASP.NET Core része, nem igényel külső csomag telepítését

6. OpenAPI + Scalar

Az API dokumentáció és interaktív tesztelőfelület. Az Microsoft.AspNetCore.OpenApi generálja az API specifikációt, a Scalar.AspNetCore pedig modern vizuális felületet nyújt a végpontok böngészéséhez és teszteléséhez.

Miért ezek lettek választva:

- Az OpenAPI automatikusan generálja az API specifikációt a controllerek és DTO-k alapján
- A Scalar a Swagger UI modern alternatívája – letisztultabb, gyorsabb felhasználói felületet biztosít
- Csak fejlesztői környezetben (**IsDevelopment()**) érhető el
- Nincs szükség Swashbuckle csomag telepítésére

7. CORS (Cross-Origin Resource Sharing)

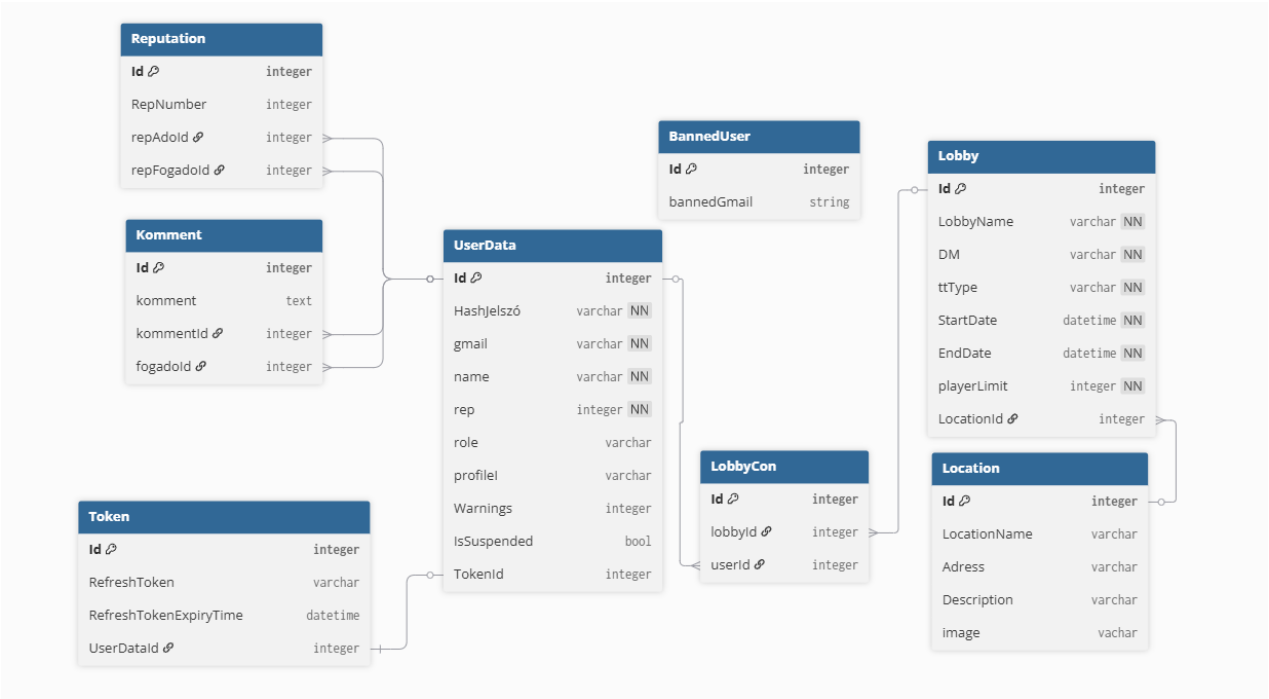
A Program.cs-ben konfigurált CORS policy lehetővé teszi, hogy a React frontend (**http://localhost:5173**) kommunikáljon a backend API-val.

Miért ez lett választva:

- A frontend és a backend különböző portokon fut, így a böngésző alapértelmezetten blokkolná a cross-origin kéréseket
- Az **AllowCredentials()** lehetővé teszi az autentikációs fejlécek küldését
- Az **AllowAnyHeader()** és **AllowAnyMethod()** biztosítja, hogy minden HTTP metódus és fejléc engedélyezve legyen

2.4.2 Adatbázis szerkezete

A weboldal nyolc adatmodellből/táblából áll, ezek SQLite adatbázisban tárolják az információkat az alábbi ábra szerint:



2.4.2.1. USER TÁBLA (UserData)

A rendszer központi táblája, amely az összes regisztrált felhasználó adatait tárolja. Tartalmazza az autentikációhoz, jogosultságkezeléshez és moderációhoz szükséges információkat. Minden más entitás közvetlenül vagy közvetve ehhez a táblához kapcsolódik.

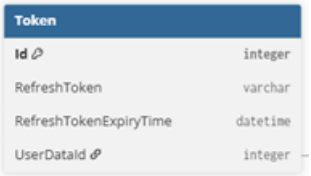
- **Name** (TEXT, NOT NULL) – A felhasználó megjelenített neve, amelyet a rendszer felületein használ
- **Hash** (TEXT, NOT NULL) – A jelszó hash-elt formája, biztonsági okokból a jelszó soha nem kerül nyers formában tárolásra
- **Gmail** (TEXT, NOT NULL) – A felhasználó email címe, amely az autentikáció és a kapcsolattartás alapja

- **Id** (Guid, PRIMARY KEY) – Egyedi azonosító, amely automatikusan generálódik a létrehozáskor
- **Role** (TEXT, NOT NULL) – A felhasználó szerepköre a rendszerben (pl. "User", "Admin"), amely meghatározza a hozzáférési jogosultságokat
- **Warnings** (INTEGER, NOT NULL) – A felhasználó által kapott figyelmeztetések száma, amelyet a moderátorok kezelnek
- **ProfileI** (TEXT, NOT NULL) – A felhasználó profilképének elérési útvonala vagy URL-je
- **IsSuspended** (INTEGER/BOOL, NOT NULL) – Jelzi, hogy a felhasználó fiókja ideiglenesen fel van-e függesztve, ami megakadályozza a bejelentkezést és a rendszer használatát
- **TokenId** (GUID, NULLABLE, FOREIGN KEY → Tokens.Id) – Az autentikációs tokenre mutató hivatkozás; nullable, mert a felhasználónak nem feltétlenül van aktív

---

2.4.2.2. TOKENS TÁBLA (Token)

A JWT-alapú autentikációt támogató tábla. Minden felhasználóhoz egy token-pár tartozik: egy rövid életű Access Token az API hívások hitelesítéséhez, és egy hosszabb életű Refresh Token, amellyel az Access Token lejárta után új token-pár kérhető anélkül, hogy a felhasználónak újra be kellene jelentkeznie. A UserData-val kölcsönös 1:1 kapcsolatban áll – mindkét oldalon van foreign key.



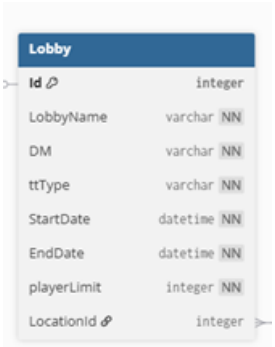
- **Id** (Guid, PRIMARY KEY) – Egyedi azonosító, amely automatikusan generálódik a létrehozáskor
- **AccesToken** (TEXT, NULLABLE) – Az aktuális Access Token értéke, amelyet a kliens minden API kérés Authorization fejlécében küld el; nullable, mert kijelentkezés után törölhető
- **AccessTokenExpiryTime** (TEXT/DATETIME, NULLABLE) – Az Access Token lejárati időpontja, amely után a token érvénytelenné válik és új token szükséges

- **RefreshToken** (TEXT, NULLABLE) – Az aktuális Refresh Token értéke, amellyel a kliens új Access Tokent igényelhet anélkül, hogy a felhasználónak újra jelszót kellene megadnia
- **RefreshTokenExpiryTime** (TEXT/DATETIME, NULLABLE) – A Refresh Token lejárati időpontja; ha ez is lejár, a felhasználónak teljes újra-bejelentkezés szükséges
- **UserDataId** (GUID, NULLABLE, FOREIGN KEY → Users.Id) – A felhasználóra mutató hivatkozás, amely meghatározza, hogy melyik

---

2.4.2.3. LOBBIES TÁBLA (Lobby)

Egy-egy szervezett társasjáték- vagy asztali szerepjáték-eseményt reprezentál. Tartalmazza az esemény összes lényeges adatát: a játék típusát, időpontját, helyszínét és létszámkorlátját. Opcionálisan kapcsolódhat fizikai helyszínhez, de helyszín nélkül is létezhet, például online játék esetén.



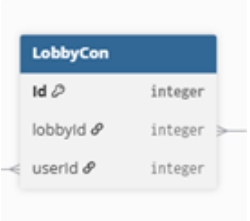
- **Id** (Guid, PRIMARY KEY) – Egyedi azonosító, amely automatikusan generálódik a létrehozáskor
- **LobbyName** (TEXT, NOT NULL) – A lobby megjelenített neve, amelyet a felhasználók a böngészés során látnak
- **Dm** (TEXT, NOT NULL) – A Dungeon Master (játékvezető) neve, aki az eseményt szervezi és vezeti
- **TtType** (TEXT, NOT NULL) – A társasjáték típusa (pl. D&D, Catan), amely segít a játékosoknak a megfelelő esemény kiválasztásában
- **Status** (TEXT, NOT NULL) – A lobby aktuális állapota (pl. nyitott, folyamatban, lezárt), amely jelzi az esemény életciklusát
- **Description** (TEXT, NOT NULL) – Az esemény részletes leírása, amelyben a szervező megadhatja a szabályokat, elvárásokat és egyéb információkat
- **locationName** (TEXT, NOT NULL) – A helyszín neve szövegként tárolva, amely gyors megjelenítést tesz lehetővé a Location tábla lekérdezése nélkül
- **StartDate** (TEXT/DATETIME, NOT NULL) – Az esemény tervezett kezdési időpontja
- **Duration** (INTEGER, NOT NULL) – Az esemény tervezett időtartama
- **PlayerLimit** (INTEGER, NOT NULL) – A maximálisan csatlakoztatható játékosok száma

- **PlayerMin** (INTEGER, NOT NULL) – A játék elindításához szükséges minimum játékosszám
- **PlayerCount** (INTEGER, NOT NULL) – Az aktuálisan csatlakozott játékosok száma, amely dinamikusan változik a csatlakozások és kilépések során
- **LocationId** (GUID, NULLABLE, FOREIGN KEY → Locations.Id) – A helyszínre mutató hivatkozás; nullable, mert az esemény lehet online vagy helyszín nélküli is

---

2.4.2.4. LOBBYCONS TÁBLA (LobbyCon)

Kapcsolótábla, amely a felhasználók és lobbyk közötti Many-to-Many kapcsolatot valósítja meg. Minden rekord egy felhasználó csatlakozását jelenti egy adott lobbyhoz. Explicit entitásként van megvalósítva saját **Id**-val, ami lehetővé teszi a későbbi bővítést (pl. csatlakozás időpontja, szerepkör a lobbyn belül).



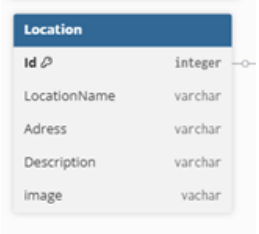
- **Id** (Guid, PRIMARY KEY) – Egyedi azonosító, amely automatikusan generálódik a létrehozáskor
- **UserDataId** (GUID, NOT NULL, FOREIGN KEY → Users.Id) – A csatlakozott felhasználó azonosítója, amely meghatározza, hogy melyik user tartozik ehhez a csatlakozáshoz

- **LobbyId** (GUID, NOT NULL, FOREIGN KEY → Lobbies.Id) – A lobby azonosítója, amelyhez a felhasználó csatlakozott

---

2.4.2.5. LOCATIONS TÁBLA (Location)

A fizikai helyszíneket tárolja, ahol a játékesemények megrendezésre kerülhetnek. Egy helyszínhez több lobby is tartozhat (1:Many kapcsolat), így ugyanazon a helyszínen több különböző esemény is szervezhető. A helyszín adatai segítenek a játékosoknak megtalálni az esemény pontos helyét.



- **Id** (Guid, PRIMARY KEY) – Egyedi azonosító, amely automatikusan generálódik a létrehozáskor
- **LocationName** (TEXT, NOT NULL) – A helyszín hivatalos neve (pl. "Társasjáték Klub Budapest")

- **Adress** (TEXT, NOT NULL) – A helyszín pontos fizikai címe, amely alapján a játékosok megtalálhatják a helyet
- **Description** (TEXT, NOT NULL) – A helyszín részletes leírása, amely tartalmazhat információkat a megközelíthetőségről, parkolásról, felszereltségről
- **Image** (TEXT, NOT NULL) – A helyszínhez tartozó kép elérési útvonala vagy URL-je, amely vizuálisan segíti az azonosítást

---

2.4.2.6. KOMMENTS TÁBLA (Komment)

A felhasználók közötti kommentelést valósítja meg. Egy felhasználó kommentet írhat egy másik felhasználó profiljára, ezzel visszajelzést adva a közös játékélményről. Két foreign key-t tartalmaz, amelyek két különböző felhasználóra mutatnak: a komment írójára és a fogadójára.

- **Id** (Guid, PRIMARY KEY) – Egyedi azonosító, amely automatikusan generálódik a létrehozáskor
- **KommentSzoveg** (TEXT, NOT NULL) – A komment szöveges tartalma, amelyet a kommentelő felhasználó írt a fogadó felhasználó profiljára
- **KommentaloUserId** (GUID, NOT NULL, FOREIGN KEY → Users.Id) – A kommentet író felhasználó azonosítója
- **FogadoUserId** (GUID, NOT NULL, FOREIGN KEY → Users.Id) – A kommentet fogadó felhasználó azonosítója, akinek a profiljára a komment kerül

---

2.4.2.7. REPS TÁBLA (Reputation)

A reputációs szavazatokat tárolja. Minden felhasználó egy másik felhasználónak egyetlen szavazatot adhat, amelynek értéke +1 (pozitív) vagy -1 (negatív). Ha a felhasználó meggondolja magát, a meglévő szavazat értéke módosul. Ez a rendszer lehetővé teszi a megbízható játékosok kiemelését a közösségben.

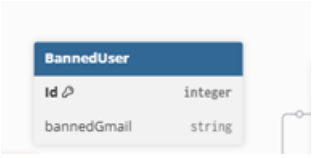
- **Id** (Guid, PRIMARY KEY) – Egyedi azonosító, amely automatikusan generálódik a létrehozáskor

- **Value** (INTEGER, NOT NULL) – A szavazat értéke: +1 vagy -1 (a migrációban RepNumber néven, a C# modellben Value)
- **RepAdoUserId** (GUID, NOT NULL, FOREIGN KEY → Users.Id) – A szavazó felhasználó azonosítója
- **RepFogadoUserId** (GUID, NOT NULL, FOREIGN KEY → Users.Id) – A szavazatot kapó felhasználó azonosítója

---

2.4.2.8. BANNEDUSERS TÁBLA (BannedUser)

Önálló, más táblához nem kapcsolódó entitás. A véglegesen kitiltott felhasználók email címeit tárolja. Regisztráció és bejelentkezés során a rendszer ellenőrzi, hogy az adott email cím szerepel-e ebben a táblában – ha igen, a hozzáférés megtagadásra kerül.



- **Id** (Guid, PRIMARY KEY) – Egyedi azonosító, amely automatikusan generálódik a létrehozáskor
- **BannedGmail** (TEXT, NOT NULL) – A kitiltott felhasználó email címe, amely alapján a rendszer blokkolja az újraregisztrálást és bejelentkezést

Kapcsolat	Típus	Leírás
UserData ↔ Token	1:1	Kölcsönös foreign key-ekkel megvalósított kapcsolat; minden felhasználóhoz legfeljebb egy aktív token-pár tartozik
UserData ↔ Lobby	Many-to-Many	A LobbyCon kapcsolótáblán keresztül; egy felhasználó több lobbyhoz csatlakozhat, és egy lobbynak több tagja lehet
UserData → Komment	1:Many (kétirányú)	Egy felhasználó több kommentet írhat (kommentelő), és több kommentet kaphat a profiljára (fogadó)
UserData → Rep	1:Many (kétirányú)	Egy felhasználó több szavazatot adhat (adó), és több szavazatot kaphat (fogadó)
Location → Lobby	1:Many	Egy helyszínhez több lobby is tartozhat, de egy lobby legfeljebb egy helyszínhez rendelhető
BannedUser	Önálló	Nincs más táblával kapcsolata

2.4.3 Api lekérések

2.4.3.1. Autentikációs Endpointok

- **POST** /api/Auth/register – Új felhasználó regisztrációja
- **POST** /api/Auth/login – Bejelentkezés (access + refresh token visszaadása)
- **POST** /api/Auth/refresh – Access token frissítése refresh tokennel

2.4.3.2. Felhasználó Endpointok

- **GET** /api/Auth/CurrentUser – Aktuális bejelentkezett felhasználó adatainak lekérdezése
- **GET** /api/Auth/Search for user(Id) – Felhasználó keresése azonosító alapján
- **GET** /api/Auth/Search for user(Name) – Felhasználó keresése név alapján
- **PATCH** /api/Auth/ChangeUserData – Felhasználói adatok módosítása (név, jelszó, profilkép)
- **GET** /api/Auth/GetLobbies_youre_in – Saját lobbik lekérdezése

2.4.3.4. Helyszín Endpointok

- **GET** /api/Auth/GetLocations – Összes helyszín listázása
- **GET** /api/Auth/GetLocation – Egy helyszín részleteinek lekérdezése
- **POST** /api/Auth/AddLocation – Új helyszín hozzáadása
- **DELETE** /api/Auth/DeleteLocation/{Id} – Helyszín törlése (a hozzá tartozó lobbik és lobby-kapcsolatok kaszkád törlésével)

2.4.3.5. Lobby Endpointok

- **GET** /api/Auth/GetAllLobbies – Összes lobby listázása a játékosok nevével együtt
- **GET** /api/Auth/GetLobby – Egy lobby részleteinek lekérdezése
- **POST** /api/Auth/CreateLobby – Új lobby létrehozása helyszínhez (időpont-ütközés ellenőrzéssel)
- **DELETE** /api/Auth/DeleteLobby/{Id} – Lobby törlése (csak a DM vagy admin)
- **POST** /api/Auth/AddPlayer – Csatlakozás lobbyhoz (létszám-ellenőrzéssel, dupla csatlakozás tiltásával)
- **DELETE** /api/Auth/LeaveLobby – Kilépés lobbyból (státusz automatikus visszaállításával)
- **DELETE** /api/Auth/RemovePlayerFromLobby – Játékos eltávolítása lobbyból (csak a DM vagy admin)

2.4.3.6. Közösségi Endpointok (Rep és Komment)

- **PATCH** /api/Auth/AddRep – Pozitív reputáció (+1) adása egy felhasználónak
- **PATCH** /api/Auth/RemoveRep – Negatív reputáció (-1) adása egy felhasználónak
- **POST** /api/Auth/WriteComment – Komment írása egy felhasználó profiljára
- **GET** /api/Auth/FogadoComments/{Id} – Egy felhasználóra érkezett kommentek lekérdezése
- **DELETE** /api/Auth/DeleteComment/{Id} – Komment törlése

2.4.3.7. Admin Endpointok

- **GET** /api/Auth/GetAllUsers – Összes felhasználó listázása (reputációval együtt)
- **PATCH** /api/Auth/ChangeToAdmin – Felhasználó adminná tétele
- **PATCH** /api/Auth/ChangeToUser – Admin visszaminősítése felhasználóvá
- **PATCH** /api/Auth/GiveWarning – Figyelmeztetés kiosztása felhasználónak
- **PATCH** /api/Auth/SuspendUser – Felhasználó felfüggesztése (minimum 3, és 3-mal osztható warningszám szükséges)
- **PATCH** /api/Auth/UnSuspendUser – Felfüggesztés feloldása
- **DELETE** /api/Auth/BanUser/{Id} – Felhasználó végleges kitiltása (összes adat kaszkád törlésével, Gmail tiltólistára helyezésével)

2.4.3.8. Egyéb Endpointok

- **GET** /api/Auth/Authorization checker – Autentikáció ellenőrzése (tesztelési célú)
- **GET** /api/Auth/admin-only – Admin jogosultság ellenőrzése (tesztelési célú)
- **PATCH** /api/Auth/Change Role Rendszergazda – Felhasználó adminná tétele autentikáció nélkül (az első admin fiók létrehozásához)

2.5 Tipikus algoritmusok bemutatása

2.5.1 Regisztrációs algoritmus

A **Register()** metódus többlépcsős validációs folyamatot hajt végre az új felhasználó létrehozása előtt. Első lépésben ellenőrzi, hogy a megadott felhasználónév vagy email cím szerepel-e már az adatbázisban az **AnyAsync()** segítségével. Ezt követően a BannedUsers táblából lekérdezi, hogy az email cím kitiltott-e. Amennyiben minden ellenőrzés sikeres, a **PasswordHasher<UserData>** osztály **HashPassword()** metódusával hash-eli a jelszót, amely automatikusan salt-olja is azt. A profilkép véletlenszerűen generálódik a **RandomProfilePic()** segédmetódussal, amely 1 és 14 közötti számot sorsol, és egy előre elkészített profilkép-útvonalat rendel hozzá. Végül a felhasználó **"User"** szerepkörrel és **IsSuspended = false** értékkel kerül mentésre.

```
[HttpPost("register")]
public  async Task<IActionResult> Register(UserDataDTO request)
{

    var number = RandomProfilePic();

    var banned=_context.BannedUsers.FirstOrDefault(x=>x.BannedGmail==request.Gmail);
    if (await _context.Users.AnyAsync(u => u.Name == request.Name))
    {
        return BadRequest("Már van ilyen felhasználó");
    }
    if (await _context.Users.AnyAsync(u => u.Gmail == request.Gmail))
    {
        return BadRequest("Már van ilyen felhasználó ilyen Gmaillel");
    }
    if (banned!=null)
    {
        return Unauthorized("Ez a Gmail ki van bannolva");
    }
    UserData ujUserData=new UserData();
    var hashedPassword=new PasswordHasher<UserData>()
        .HashPassword(ujUserData,request.Password);
    ujUserData.Id=Guid.NewGuid();
    ujUserData.Name=request.Name;
    ujUserData.Hash = hashedPassword;
    ujUserData.Gmail=request.Gmail;
    ujUserData.Role="User";

    ujUserData.ProfileI = $"/assets/profilepics/random/RandomProfilePic{number}.png";
    ujUserData.IsSuspended = false;
    if(ujUserData.Name==""|| ujUserData.Gmail == "" || ujUserData.Hash == "" )
    {
        return BadRequest("Név, Gmail vagy jelszó hiányzik");
    }
    _context.Users.Add(ujUserData);
    await _context.SaveChangesAsync();
    return Ok(new { ujUserData.Id, ujUserData.Name, ujUserData.Gmail, ujUserData.Role,ujUserData.ProfileI});

}
```

2.5.2 JWT Token generálás

A **CreateToken()** metódus a felhasználó adataiból JSON Web Tokent állít elő. Először egy **Claim** listát épít fel, amely tartalmazza a felhasználó nevét (ClaimTypes.Name), egyedi azonosítóját (**ClaimTypes.NameIdentifier**) és szerepkörét (ClaimTypes.Role).
Az appsettings.json-ből beolvassa a titkos kulcsot, amelyből **SymmetricSecurityKey**-t hoz létre, majd **HmacSha512** aláírási algoritmussal **SigningCredentials** objektumot képez. Végül a **JwtSecurityToken** konstruktorral összeállítja a tokent – beállítva az issuer-t, audience-t, a claims listát, a lejárati időt és az aláírást –, amelyet a **JwtSecurityTokenHandler.WriteToken()** metódus sorosít szöveges formátumba.

```
private string CreateToken(UserData user) {

    var claims = new List<Claim>
    {
        new Claim(ClaimTypes.Name,user.Name),
        new Claim(ClaimTypes.NameIdentifier,user.Id.ToString()),
        new Claim(ClaimTypes.Role,user.Role)
    };
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration.GetValue<string>("Appsettings:Token")!));
    var creds=new SigningCredentials(key,SecurityAlgorithms.HmacSha512);
    var tokenDescriptor = new JwtSecurityToken(
        issuer:_configuration.GetValue<string>("Appsettings:Issuer"),
        audience: _configuration.GetValue<string>("Appsettings:Audience"),
        claims:claims,
        expires:DateTime.UtcNow.Add(_accessTokenLifetime),
        signingCredentials:creds
        );
    return new JwtSecurityTokenHandler().WriteToken(tokenDescriptor);
}
```

2.5.3 Lobby létrehozása időpont-ütköző ellenőrzéssel

A **CreateLobby()** metódus az alapvető validáción túl (helyszín létezése, minimum > maximum játékosszám ellenőrzés) egy időintervallum-átfedés algoritmust alkalmaz. Az adatbázisból lekérdezi, hogy az adott helyszínen van-e olyan meglévő lobby, amelynek időintervalluma átfedi az újonnan kértet. Az átfedés matematikai feltétele: két **[A_start, A_end]** és **[B_start, B_end]** intervallum akkor fedi át egymást, ha **A_start <= B_end ÉS B_start <= A_end**. A kódban ez a StartDate és a Duration (óra) alapján számolt végidőponttal valósul meg.

```
var reserved = await _context.Lobbies.FirstOrDefaultAsync(x => x.LocationId == locationId.Id &&
x.StartDate <= request.StartDate.AddHours(request.Duration) &&
request.StartDate <= x.StartDate.AddHours(x.Duration));

if (reserved != null)
{
    return BadRequest("Ezen a helyen és időpontban már van egy foglalt lobby!");
}
```

2.5.4 Játékos csatlakozása lobbyhoz

Az **AddPlayer()** metódus négy egymásra épülő üzleti szabályt ellenőriz a csatlakozás engedélyezése előtt. Először megvizsgálja, hogy a kérelmező nem a lobby DM-je (játékvezetője), mivel a szervező automatikusan a lobby tagja. Másodszor a LobbyCons táblából **AnyAsync()** hívással ellenőrzi a dupla csatlakozás tiltását. Harmadszor összehasonlítja az aktuális PlayerCount értéket a PlayerLimit-tel. Ha minden feltétel teljesül, növeli a játékosszámot és létrehozza a LobbyCon kapcsoló-rekordot. Végül automatikus státuszváltást hajt végre: ha a PlayerCount eléri a PlayerMin értéket, a lobby státusza **"Pending"**-ről **"Confirmed"**-ra változik.

```
if (lobby.Dm == user.Name)
{
    return BadRequest("A DM nem csatlakozhat a saját lobbyjához.");
}
var alreadyInLobby = await _context.LobbyCons
    .AnyAsync(x => x.LobbyId == Id && x.UserDataId == user.Id);
if (alreadyInLobby)
{
    return BadRequest("Már csatlakoztál ehhez a lobbyhoz.");
}

if (lobby.PlayerCount >= lobby.PlayerLimit)
{
    return BadRequest("A lobby elérte a maximum játékosszámot.");
}
lobby.PlayerCount++;
var newLobbyCon = new LobbyCon
{
    Id = Guid.NewGuid(),
    UserDataId = user.Id,
    LobbyId = lobby.Id
};
_context.LobbyCons.Add(newLobbyCon);
_context.Lobbies.Update(lobby);
await _context.SaveChangesAsync();
if (lobby.PlayerCount >= lobby.PlayerMin)
{
    lobby.Status = "Confirmed";
    _context.Lobbies.Update(lobby);
    await _context.SaveChangesAsync();
}
return Ok(new());
```

2.6 Tesztdokumentáció

A program tesztelése során a következő eseteket vizsgáltuk:

- ✓ Különböző böngészőkbeli működés (Microsoft Edge, Google Chrome)
- ✓ Weboldal reszponzív teszt
- ✓ Adatbázis/backend hibakezelés

QuestBook oldal fejlesztése során némi hibákba ütköztünk, amiket sikeresen kijavítottunk:

1. hiba: Az authentikáció refresh tokenje frissül az access tokennel együtt

Leírás: Amikor a refresh endpointot használtam a Scalar API-ban, a refresh token az access tokennel együtt lett frissítve, miközben csak az access tokennek kellett volna frissülnie.
Lehetséges ok: A GenAndSaveRefreshTokenAsync metódus a hozzáférési token frissítésekor a refresh tokent is újragenerálta, miközben az eredetit kellett volna meghagynia. A refresh token kizárólag a login endpoint használatakor kellett volna, hogy frissüljön.

Megoldás: A GenAndSaveRefreshTokenAsync metódust úgy módosítottam, hogy az a meglévő refresh tokent és annak lejárati idejét változatlanul hagyja, és csak visszaadja az eredeti értéket ahelyett, hogy újat generálna.

```
public async Task<TokenDTO?> RefreshTokenAsync( string refreshToken)
{
    var token = await ValidateRefreshTokenAsync(refreshToken);
    if (token == null) return null;
    var user = await _context.Users.FirstOrDefaultAsync(x => x.Id == token.UserDataId);
    if (user == null) return null;
    var newAccesToken = CreateToken(user);
    token.AccesToken = newAccesToken;
    token.AccessTokenExpiryTime = DateTime.UtcNow.Add(_accessTokenLifetime);

    var newRefreshToken = await GenAndSaveRefreshTokenAsync(token);
    return new TokenDTO
    {
        AccesToken = newAccesToken,
        AccessTokenExpiryTime = token.AccessTokenExpiryTime,
        RefreshToken = refreshToken,
        RefreshTokenExpiryTime = token.RefreshTokenExpiryTime

    };
```

2. hiba: A frontend és a backend nincs összekapcsolva

Leírás: Hiába volt megadva a helyes API-cím a frontenden (**https://localhost:7231**), a böngésző CORS-hiba miatt blokkolta a kéréseket, így a frontend nem tudott adatot küldeni a backendnek.

Lehetséges okok:

- Ellenőriztem, hogy a helyes API-link van-e megadva a frontenden.
- A backend szerver futás közben nem adott explicit hibát, de a böngésző konzoljában CORS-hibaüzenet jelent meg.

Megoldás: A backend Program.cs fájljából hiányzott a CORS szolgáltatás konfigurálása. Hozzáadtam az **AddCors** szolgáltatást a **http://localhost:5173** origin engedélyezésével, valamint a **UseCors** middleware-t a kérés-feldolgozási pipeline-ban.

```
builder.Services.AddCors(options =>
{
    options.AddPolicy(name:"test", policy =>
    {
        policy.WithOrigins("http://localhost:5173")
              .AllowAnyHeader()
              .AllowAnyMethod()
              .AllowCredentials();
    });
});
```

3. hiba: A SuspendUser endpoint feltételei nem működnek

Leírás: Amikor egy felhasználónak több mint három figyelmeztetése van, és hárommal osztható a figyelmeztetések száma, akkor a felhasználó fiókja felfüggeszthető; más esetben BadRequestet kellene visszadobnia. Viszont az endpoint az ellenkezőjét hajtotta végre a legtöbb esetben.

Lehetséges okok:

- A feltételben hibásan voltak megadva a relációs jelek.
- A hárommal maradék nélküli osztásos feltétel hibásan lett leírva.

Megoldás: Az endpointban a Warnings mezőre vonatkozó relációs jel rossz irányba nézett (pl. > volt < helyett). A javítás után a feltétel **suspendedUser.Warnings < 3** lett, ami helyesen elutasítja a felfüggesztést, ha a felhasználónak kevesebb mint 3 figyelmeztetése van. A hárommal való oszthatóság feltétele (**Warnings % 3 != 0**) szintén korrigálva lett.

```
[Authorize(Roles = "Admin")]
[HttpPatch("SuspendUser")]// működik
public async Task<IActionResult> SuspendUser([FromQuery]Guid Id)
{
    var check = await ValidateAccesToken();
    if (check != null)
    {
        return check;
    }
    var suspendedUser = _context.Users.FirstOrDefault(x => x.Id == Id);

    if (suspendedUser == null)
    {
        return BadRequest("Nincs ilyen felhasználó");
    }
    if (suspendedUser.Warnings < 3)
    {
        return BadRequest("Ennek a felhasználónak nincs elég warningja");
    }
    if (suspendedUser.Warnings % 3 != 0)
    {
        return BadRequest("Csak minden harmadik figyelmeztetésnél lehet felfüggezteni felhasználót ");
    }
    suspendedUser.IsSuspended = true;
    _context.Users.Update(suspendedUser);
    await _context.SaveChangesAsync();
    return Ok();
}
```

4. hiba: A JWT token RoleClaim-je nem változik, amikor a felhasználó jogosultsága megváltozik

Leírás: Ha egy felhasználó Admin rangot kapott, a JWT tokenben tárolt role claim továbbra is „User" maradt, így az admin jogosultságot igénylő endpointokhoz nem kapott hozzáférést.
Lehetséges okok:

- Az adminos endpointokban rosszul küldtem be a felhasználó adatait.
- Amikor megváltoztatom a felhasználó jogosultságát adminra az adatbázisban, az adatbázisban tárolt access token továbbra is a régi role claimmel rendelkező JWT-t tartalmazta, nem pedig az új jogosultsággal generáltat.
-

Megoldás: A ChangeRoleAdmin és ChangeRoleUser endpointokban a role módosítása után a CreateToken metódus meghívásával új access tokent generálok, amely már az aktualizált role claimet tartalmazza. Ezt az új tokent az adatbázisban is frissítem, miközben a refresh token változatlan marad. Ez szintén igaz, ha egy admint lefokozunk felhasználóra.

```
[Authorize(Roles = "Admin")]
[HttpPatch("ChangeToAdmin")]// működik
public async Task<ActionResult<CurrentUserDTO>> ChangeRoleAdmin(Guid Id)
{
    var check = await ValidateAccesToken();
    if (check != null)
    {
        return check;
    }
    var currentId = User?.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    var currentRole = User?.FindFirst(ClaimTypes.Role)?.Value;
    var changedUser = _context.Users?.FirstOrDefault(x => x.Id == Id);
    var roleChanger = _context.Users?.FirstOrDefault(x => x.Id.ToString() == currentId && currentRole == "Admin");
    if (changedUser == null)
    {
        return BadRequest("Nincs ilyen felhasználó");
    }
    if (currentRole != "Admin")
    {
        return Unauthorized("Ehhez nincs jogod!");
    }
    changedUser.Role = "Admin";
    _context.Users.Update(changedUser);
    await _context.SaveChangesAsync();
    var newAccessToken = CreateToken(changedUser);
    var tokenRow = await _context.Tokens.FirstOrDefaultAsync(t => t.UserDataId == changedUser.Id);

    if (tokenRow != null)
    {
        tokenRow.AccesToken = newAccessToken;
        tokenRow.AccessTokenExpiryTime = DateTime.UtcNow.Add(_accessTokenLifetime);
        _context.Tokens.Update(tokenRow);

        var newRefreshToken = await GenAndSaveRefreshTokenAsync(tokenRow);

        await _context.SaveChangesAsync();

        return Ok(new TokenDTO
        {
            AccesToken = newAccessToken,
            AccessTokenExpiryTime = tokenRow.AccessTokenExpiryTime,
            RefreshToken = newRefreshToken,
            RefreshTokenExpiryTime = tokenRow.RefreshTokenExpiryTime
        });
    }
    return Ok();
```

```
[Authorize(Roles = "Admin")]
[HttpPatch("ChangeToUser")]// működik
public async Task<ActionResult<CurrentUserDTO>> ChangeRoleUser( Guid Id)
{
    var check = await ValidateAccesToken();
    if (check != null)
    {
        return check;
    }
    var currentId = User?.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    var currentRole = User?.FindFirst(ClaimTypes.Role)?.Value;
    var changedUser = _context.Users?.FirstOrDefault(x => x.Id == Id);
    var roleChanger = _context.Users?.FirstOrDefault(x => x.Id.ToString() == currentId && currentRole== "Admin");
    if (changedUser == null)
    {
        return BadRequest("Nincs ilyen felhasználó");
    }
    if (currentRole!= "Admin")
    {
        return Unauthorized("Ehhez nincs jogod!");
    }
    changedUser.Role = "User";
    _context.Users.Update(changedUser);
    await _context.SaveChangesAsync();
    var newAccessToken = CreateToken(changedUser);
    var tokenRow = await _context.Tokens.FirstOrDefaultAsync(t => t.UserDataId == changedUser.Id);

    if (tokenRow != null)
    {
        tokenRow.AccesToken = newAccessToken;
        tokenRow.AccessTokenExpiryTime = DateTime.UtcNow.Add(_accessTokenLifetime);
        _context.Tokens.Update(tokenRow);

        var newRefreshToken = await GenAndSaveRefreshTokenAsync(tokenRow);

        await _context.SaveChangesAsync();

        return Ok(new TokenDTO
        {
            AccesToken = newAccessToken,
            AccessTokenExpiryTime = tokenRow.AccessTokenExpiryTime,
            RefreshToken = newRefreshToken,
            RefreshTokenExpiryTime = tokenRow.RefreshTokenExpiryTime
        });
    }
    return Ok();
}
```

2.7 Frontend ismertetése

A frontend Visual Studio Code környezetben készült, TypeScript nyelven, React 19 keretrendszer és Vite build eszköz felhasználásával. Az alkalmazás egyoldalas (SPA – Single Page Application) architektúrát követ, ahol a teljes felhasználói felület egy központi komponensből (App.tsx) kerül vezérlésre.

A nézetek közötti váltás nem külső routing könyvtár segítségével történik, hanem belső állapotkezeléssel, amely az alkalmazás egyszerűbb és gyorsabban átlátható felépítését tette lehetővé.

A frontend a backend REST API-val HTTP kéréseken keresztül kommunikál, a JWT tokenek Authorization fejlécben történő továbbításával. Az adatok feldolgozása és megjelenítése több lépésben történik, amely során a backend válaszok átalakításra kerülnek a frontend számára megfelelő formátumba.

2.7.1 Frontendhez használt technológiák

1. **React 19 (TypeScript)**

A felhasználói felület React funkcionális komponensekkel került megvalósításra.

Miért ez lett választva:

- Komponens alapú felépítés → újrafelhasználható elemek
- Hook rendszer (useState, useEffect) → állapotkezelés és adatbetöltés
- Széles körű dokumentáció és közösségi támogatás

A fejlesztés során a hook-ok használata több esetben nehézséget okozott, különösen az állapotfrissítések és újrarenderelések kezelésénél.

2. **Vite**

A frontend build és fejlesztői környezet biztosítására szolgál.

Miért ez lett választva:

- Gyors fejlesztői szerver
- Egyszerű konfiguráció
- Modern JavaScript/TypeScript támogatás

A gyors újratöltés jelentősen segítette a fejlesztési folyamatot, különösen hibakeresés során.

3. **CSS (egyedi stílusok)**

A felület kialakítása egyedi CSS segítségével történt.

Miért ez lett választva:

- Teljes kontroll a megjelenés felett
- Egyszerű integráció a komponensekkel
- Külső könyvtárak elkerülése

A gyakorlatban több elrendezési probléma is felmerült (pl. elcsúszó elemek, nem megfelelő igazítás), amelyek többszöri módosítással kerültek javításra.

2.7.2 Állapotkezelés

Az alkalmazás állapotkezelése a React hook rendszerére épül.

A legfontosabb hook-ok:

- useState – nézetek, kiválasztott elemek és felhasználói adatok kezelése
- useEffect – API hívások és adatbetöltés
- useMemo – listaelemek rendezése és optimalizálása

Az állapotkezelés során több probléma is jelentkezett, például:

- nem frissülő UI elemek
- hibás adatmegjelenítés
- nem várt újrarenderelések

Ezek a hibák gyakran több komponens együttes működéséből adódtak, ezért megoldásuk több lépéses hibakeresést igényelt.

2.7.3 Backend kommunikáció

A frontend és backend közötti kommunikáció HTTP kérésekkel történik.

A kommunikáció főbb jellemzői:

- JWT token használata (Authorization header)
- központi apiFetch függvény
- egységes hibakezelés

```
useEffect(() => {
async function loadSessions() {
  try {
    if (loggedInUser) {
      const myApiLobbies = await getMyLobbies();
      setMySessions(myApiLobbies.map(mapApiLobbyToSession));
    } else {
      setMySessions([]);
    }

    if (activeTab === "community") {
      const communityApiLobbies = await getAllLobbies();
      setCommunitySessions(communityApiLobbies.map(mapApiLobbyToSession));
    }
  } catch (error) {
    console.error("Sessions load error:", error);
  }
}

loadSessions();
```

Az API válaszok és a frontend adattípusok közötti különbségek miatt mapper függvények kerültek bevezetésre.

Feladatuk:

- API válaszok átalakítása frontend modellekké
- hiányzó vagy eltérő mezők kezelése
- típusbiztonság biztosítása

A fejlesztés során több alkalommal előfordult, hogy a backend válasz nem a várt formában érkezett, ami további átalakításokat tett szükségessé.

```
export function mapApiCurrentUserToUser(apiUser: CurrentUserResponse): User {
  return {
    id: apiUser.id,
    name: apiUser.name,
    image:
      apiUser.imageUrl && apiUser.imageUrl.trim() !== ""
        ? apiUser.imageUrl
        : getFallbackProfilePic(apiUser.name),
    rep: apiUser.rep ?? 0,
    role: apiUser.role,
    comments: [],
  };
}

Tabnine | Edit | Test | Explain | Document
export function mapSearchUserToUser(
  apiUser: SearchUserResponse,
  fallbackId: string
): User {
  return {
    id: apiUser.id || fallbackId,
    name: apiUser.name,
    image:
      apiUser.profileI && apiUser.profileI.trim() !== ""
        ? apiUser.profileI
        : apiUser.profileIUrl && apiUser.profileIUrl.trim() !== ""
        ? apiUser.profileIUrl
        : getFallbackProfilePic(apiUser.name),
    rep: apiUser.rep ?? 0,
    role: apiUser.role,
    comments: [],
  };
}
```

2.7.4 Komponens struktúra

A frontend több különálló komponensből épül fel, például:

- LocationCard
- SessionCard
- ProfilePage
- ReservationDetailsCard

A komponens alapú felépítés javította az átláthatóságot és az újrafelhasználhatóságot. Ugyanakkor az adatok átadása több esetben nehezen követhetővé vált, különösen több szintű komponenshierarchia esetén.

```
export default function SessionDetailsCard({
  session,
  onBack,
  onLeave,
  onDeleteLobby,
  onKick,
  currentUserName,
  currentUserId,
}: SessionDetailsCardProps) {
  const currentPlayerCount = session.players.length;
  const isDm =
    (currentUserName != null && session.dmName === currentUserName) ||
    (currentUserId != null && session.dmName === currentUserId);

  return (
    <section className="session-details-card">
      <div
        className="session-details-left-bar"
        style={{ backgroundColor: session.systemColor }}
      />

      <div className="session-details-content">
        <h2 className="session-details-title">
          {session.title} - Session {session.sessionNumber}
        </h2>

        <div className="session-details-system-row">
          <span
            className="session-details-system"
            style={{ color: session.systemColor }}
          >
            {session.system}
          </span>

          <span
            className={`session-details-status ${
              session.status === "confirmed"
                ? "status-confirmed"
                : "status-pending"
            }`}
          >
            {session.status === "confirmed" ? "Confirmed" : "Pending"}
          </span>
        </div>

        <div
          className="session-details-divider"
          style={{ backgroundColor: session.systemColor }}
        />
```

2.7.5 Felhasználói felület kialakítása

A felület célja egy egyszerűen kezelhető és átlátható rendszer létrehozása volt.

A fejlesztés során felmerült problémák:

- elcsúszó layout elemek
- eltérő megjelenés különböző felbontásokon
- nem megfelelően működő stílusok

Ezek javítása iteratív módon történt, több lépésben finomítva a felület megjelenését.

2.7.6 Fejlesztési nehézségek

A frontend fejlesztése során több probléma is felmerült:

- állapotkezelési nehézségek
- dátumkezelési hibák
- API és frontend közötti eltérések
- nem egyértelmű működési hibák

Bizonyos esetekben a hibák pontos oka nem volt egyértelmű, azonban a működő megoldás eléréséig több alternatíva kipróbálásra került.

2.8 Frontend fejlesztés során szerzett tapasztalatok

A frontend fejlesztése során szerzett tapasztalatok jelentősen hozzájárultak a modern webalkalmazások működésének megértéséhez.

2.8.1 Hibakeresés

A fejlesztés során a hibakeresés kiemelt szerepet kapott.
Alkalmazott módszerek:

- konzolos logolás
- böngésző DevTools
- hálózati kérések vizsgálata

A hibák megoldása gyakran több lépéses folyamat volt, amely során több lehetséges megoldás kipróbálása is szükséges volt.

2.8.2 Kódszerkezet

A projekt előrehaladtával a kódszerkezet folyamatosan javult.
Főbb változások:

- komponensekre bontás
- service réteg bevezetése
- mapper függvények használata

Ez javította az átláthatóságot és a karbantarthatóságot, bár a teljes optimalizálás nem minden esetben volt megvalósítható.

2.8.3 Felhasználói élmény

A fejlesztés során fontos szempont volt a felhasználói visszajelzés biztosítása.

Megvalósított elemek:

- hibaüzenetek megjelenítése
- státusz visszajelzések
- vizuális jelölések

Ez hozzájárult a rendszer használhatóságának javításához.

2.8.4 Összegzés

A frontend fejlesztése során több olyan probléma is felmerült, amelynek megoldása hosszabb időt és kísérletezést igényelt.

A projekt végére azonban sikerült egy stabilan működő rendszert létrehozni, amely megfelel a kitűzött céloknak és képes együttműködni a backend rendszerrel.

2.9 További fejlesztési lehetőségek

1. **Valós idejű értesítések SignalR WebSocket integrációval** – Jelenleg a felhasználók csak a felület manuális frissítésekor látják a változásokat (pl. új játékos csatlakozása, lobby státuszváltás). A SignalR bevezetésével a szerver azonnal értesíthetné a klienseket minden releváns eseményről, így a felhasználók valós időben kapnának push értesítéseket a lobbyhoz csatlakozásokról, kommentekről és reputáció-változásokról.
2. **Térképes helyszín-megjelenítés** – A helyszínek jelenleg szöveges cím formájában jelennek meg. Egy Leaflet vagy Google Maps integráció lehetővé tenné a helyszínek vizuális megjelenítését interaktív térképen, ahol a felhasználók a közelükben lévő játékhelyeket könnyedén megtalálhatnák, és közvetlenül a térképről navigálhatnának a helyszín részleteire.

3. Felhasználói dokumentáció

3.1 A program rövid bemutatása

A **QuestBook** egy közösségi webes platform és hozzá tartozó asztali adminisztrációs alkalmazás, amely lehetővé teszi asztali társasjáték- és szerepjáték-események (TTRPG – Tabletop Role-Playing Game) szervezését, helyszínek böngészését, csapatkeresést, valamint a résztvevők értékelését.

A felhasználók a böngészőjükből érhetik el a weboldalt, ahol regisztráció után lobbikat (játékalkalmakat) hozhatnak létre, csatlakozhatnak meglévő eseményekhez, és visszajelzést adhatnak egymásnak. Az adminisztrátorok egy külön WPF asztali alkalmazáson keresztül kezelhetik a felhasználókat, lobbikat és helyszíneket.

A weboldal angol nyelvű, hogy szélesebb felhasználói kör elérését teszi lehetővé, amely nemcsak magyar, hanem külföldi felhasználók számára is könnyen használható.

A rendszer három fő komponensből áll:

Webes felület (frontend) – a felhasználók számára elérhető böngészős alkalmazás

Szerveroldali API (backend) – az adatkezelés és üzleti logika

Asztali adminisztrációs panel – a rendszergazdák számára készült WPF alkalmazás

3.2 Rendszerkövetelmények (hardver)

A program használatához az alábbi hardver szükséges:

Komponens	Minimum	Ajánlott
Operációs rendszer	Windows 10 (x64) – az admin panel WPF-alapú, ezért kizárólag Windows	Windows 11 (x64)
Processzor	1.6 GHz, 64 bites, 2 mag	Intel i5 (8. gen.+) vagy egyenértékű, 4 mag
RAM	4 GB	8 GB
Tárhely	2 GB szabad lemezterület	5 GB SSD
Képernyőfelbontás	1366×768 pixel	1920×1080 pixel (Full HD)
Hálózat	Internetkapcsolat (a webes felülethez)	Stabil szélessávú internetkapcsolat

Megjegyzés: A webes felület önmagában operációsrendszer-független – bármilyen modern böngészőből elérhető (Windows, macOS, Linux). Az admin panel használatához viszont Windows operációs rendszer szükséges.

3.3 Szoftverkövetelmények

- ✓ A webes felület használatához:
  Modern böngésző: Google Chrome (120+), Microsoft Edge (120+), Firefox (120+) vagy Safari (17+) és JavaScript engedélyezve a böngészőben
- ✓ Az admin panel használatához:
  Windows 10 vagy 11 (x64) és .NET 10 Desktop Runtime telepítve (letölthető:https://dotnet.microsoft.com/download)
- ✓ A teljes rendszer futtatásához (fejlesztői/lokális mód):
  .NET 10 SDK (backend + admin panel) és Node.js 18+ és npm (frontend)

3.4 Telepítés és indítás

1. **lépés: Forráskód letöltése**
   - A GitHub repository klónozása:

   · git clone https://github.com/SzakacsMate/Vizsgaremek_SzakacsM_BarnaIVagy a ZIP archívum letöltése a GitHub felületéről.

2. **lépés: Backend indítása**
   - Nyissa meg a backendSzM.sln fájlt Visual Studio 2022-ben.
   - Nyomja meg a **Ctrl+F5** billentyűkombinációt a szerver indításához.
   - A backend elérhető lesz a **https://localhost:7231** címen.
   - Az API dokumentáció fejlesztői módban megtekinthető: **https://localhost:7231/scalar/v1**.
3. **lépés Frontend indítása**
   - Nyisson egy terminált a frontend mappában.
   - Futtassa: npm install (csak az első alkalommal szükséges).
   - Futtassa: npm run dev.
   - A weboldal elérhető lesz a http://localhost:5173 címen.
4. **lépés: Admin panel indítása**
   - Nyissa meg a QuestBook_AdminPanel_SzM.sln fájlt Visual Studio 2022-ben.

- Nyomja meg a Ctrl+F5 billentyűkombinációt.
- A bejelentkezési ablak megjelenik; admin jogosultságú felhasználó adataival lépjen be.

Fontos: A backend szervernek futnia kell az admin panel és a weboldal használatához egyaránt, mivel mindkét kliens a **https://localhost:7231** címen lévő API-t használja.

3.5 A program részletes használata

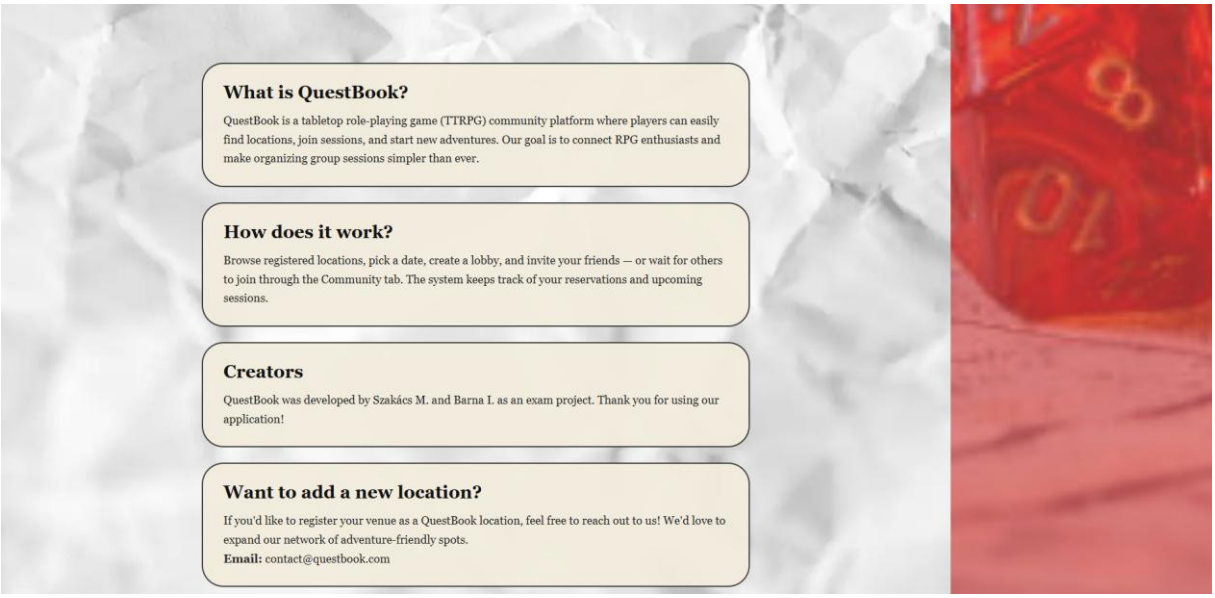
Az alábbi fejezetek részletesen ismertetik a webes felület minden elérhető funkcióját. A felhasználóbarát, egyoldalas alkalmazás (SPA) a következő fő navigációs menüpontokkal rendelkezik:

- **Locations** – Helyszínek böngészése (**bejelentkezés nélkül is elérhető**)
- **My Sessions** – Saját lobbik listázása (csak bejelentkezve)
- **Community** – Összes elérhető lobby böngészése (csak bejelentkezve)
- **About us** – Az alkalmazásról szóló információs oldal

3.5.1 Főoldal – About us

A weboldal megnyitásakor az alábbi kezdőképernyő fogadja a felhasználót, ahol a felhasználók beléphetnek vagy regisztrálhatnak az oldalra.

A kezdőlapot legörgetve az “**About us”** oldal jelenik meg.
Így bejelentkezés/regisztrálás nélkül is megismerkedhet a weboldallal, céljával, működésével, a készítőkkel és rövid ismertetőt kap arról, hogyan adhat hozzá új helyszíneket.



3.5.2 Bejelentkezés:

A **regisztrált felhasználók** a jobb oldali sávban a **„LOGIN"** gombra léphetnek be az oldalra. A bejelentkezési felület teljes képernyős nézetben jelenik meg, jobb oldalon a háttérképpel, bal oldalon az űrlappal.

Beviteli mezők:

Mező	Elvárt formátum	Kötelező
Username	A regisztrációkor megadott felhasználónév. Pontos egyezés szükséges (kis-/nagybetű érzékeny). Ékezetes karakterek megengedettek.	Igen
Email Address	A regisztrációkor megadott email cím. Érvényes email formátum szükséges (**type="email"**).	Igen
Password	A regisztrációkor megadott jelszó. Rejtett megjelenítés. Pontos egyezés szükséges.	Igen

Lehetséges hibaüzenetek:

·„User not found" – a megadott név és email kombináció nem található.

·„Wrong password" – helytelen jelszó.

·„This user is suspended!" – a fiók ideiglenesen felfüggesztve (admin döntés).

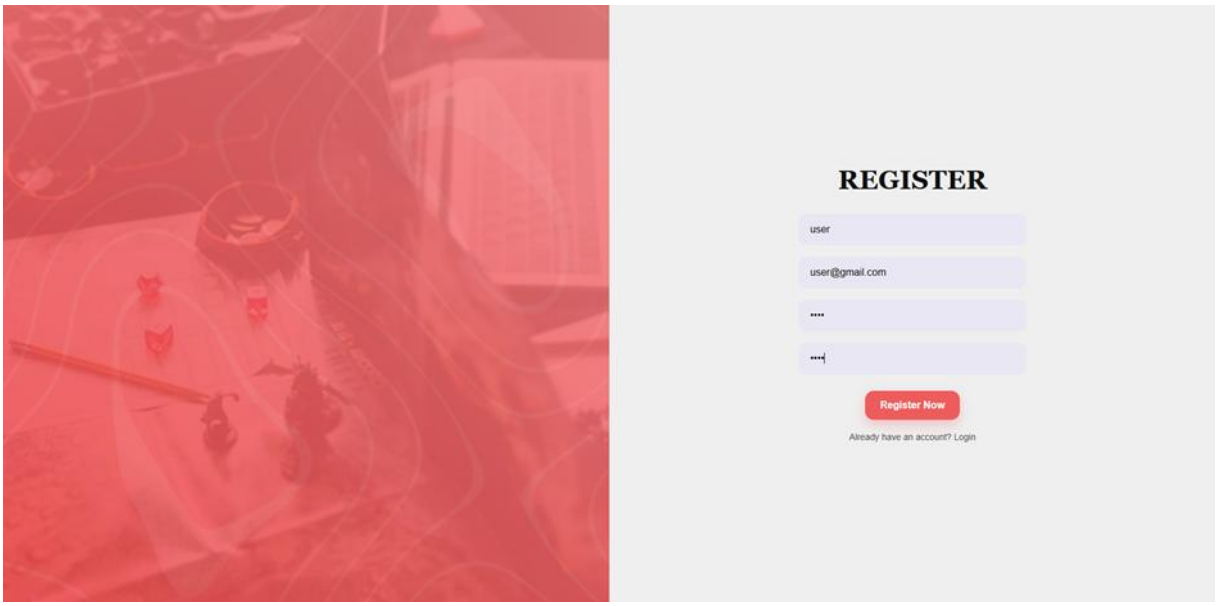
·„This gmail has been banned" – az email véglegesen tiltva.

·„*Login / register failed."* – általános hiba (piros sáv a fejlécben).

Sikeres bejelentkezés után a navigációs sáv jobb oldalán megjelenik a profilkép, és elérhetővé válnak a bejelentkezett felhasználói funkciók (My Sessions, Community).

3.5.3 Regisztráció

Új felhasználók a jobb oldali sávban a **"REGISTER"** gombra kattintva tudnak regisztrálni az oldalra. Ezt követően a regisztrációs felület teljes képernyős nézetben jelenik meg, bal oldalon egy tematikus háttérképpel, jobb oldalon az űrlappal.



Beviteli mezők:

Mező	Elvárt formátum	Kötelező
Username	Szöveges mező, max. 255 karakter. Egyedi felhasználónév. Ékezetes karakterek (á, é, í, ó, ö, ő, ú, ü, ű) megengedettek. Kis- és nagybetűk egyaránt használhatók. Nem lehet üres. Szóköz és speciális karakterek megengedettek.	Igen
Email Address	Érvényes email formátum (pl. **nev@gmail.com**). A böngésző automatikusan ellenőrzi az email formátumot (**type="email"**). Egyedi – nem lehet két azonos email a rendszerben. Kitiltott email címmel nem lehet regisztrálni. Kis- és nagybetűk, számok, pont, kötőel megengedettek.	Igen
Password	Szöveges mező (rejtett megjelenítés). Nem lehet üres. Kis- és nagybetűk, számok, speciális karakterek (**!@#$%^&*** stb.) egyaránt használhatók. Nincs minimális hossz megkötés. Ékezetes karakterek megengedettek.	Igen
Password Again	Meg kell egyeznie a **Password** mezővel. Azonos formátumi elvárások.	Igen

Lehetséges hibaüzenetek:

·„There's an user with this name already" – a megadott felhasználónév foglalt.

·„There's a user already with this gmail" – az email cím már regisztrálva van.

·„This gmail has been banned" – a megadott email véglegesen tiltva van.

·„Missing username,password or gmail" – valamelyik kötelező mező üres.

·*„Passwords do not match."* – a két jelszó mező nem egyezik (frontend ellenőrzés).

·*„Login / register failed."* – általános hiba a regisztráció során (piros sáv a fejlécben).

·

Sikeres regisztráció után az alkalmazás **automatikusan bejelentkezteti** a felhasználót, és a navigációs sávban megjelenik a profilkép.

3.5.4 Bejelentkezett felhasználók funkciói

Navigációs menüpontok (bejelentkezés után):

- **Locations** – Helyszínek böngészése (mindenki számára elérhető)
- **My Sessions** – Saját lobbik listázása (csak bejelentkezve)
- **Community** – Összes elérhető lobby böngészése (csak bejelentkezve)
- **About us** – Az alkalmazásról szóló oldal

Kijelentkezett állapotban a **Locations**, **My Sessions** és **Community** menüpontok szürkén, letiltva jelennek meg.

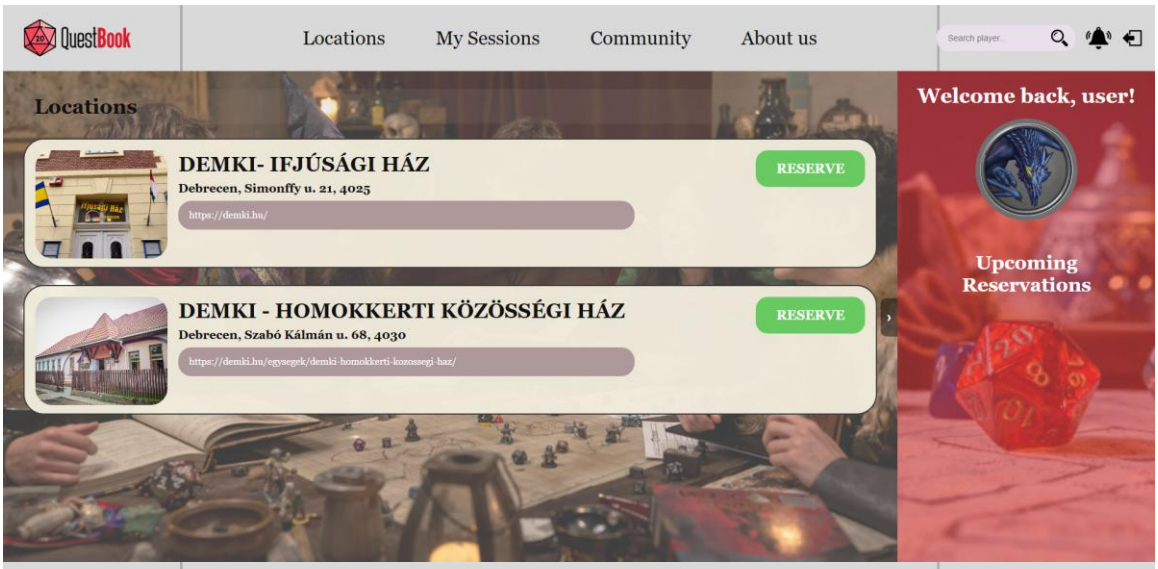
3.5.5 Helyszínek böngészése

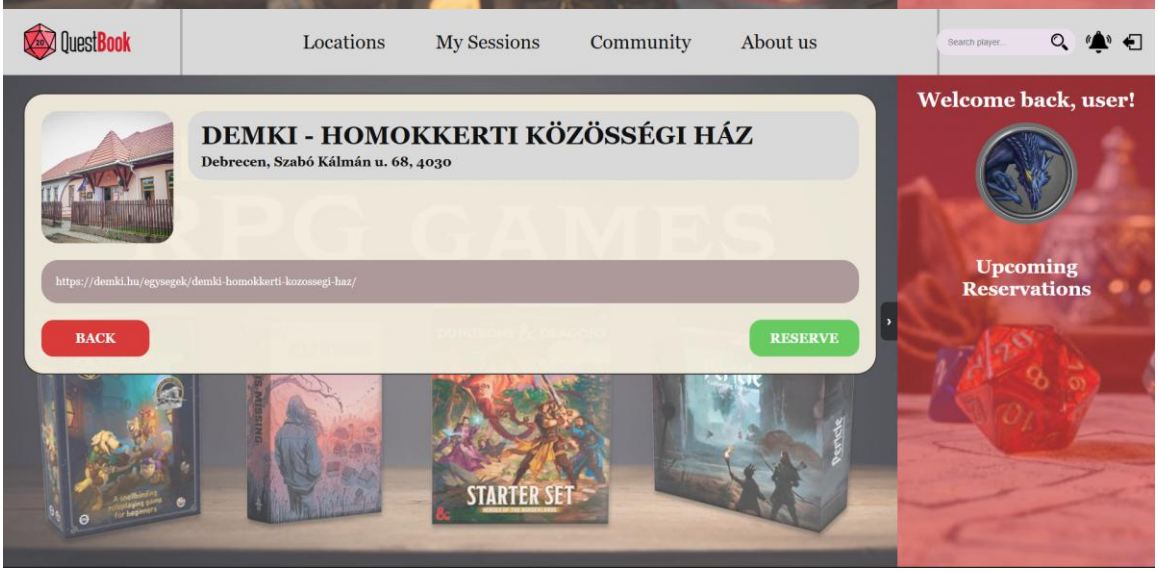
A **Locations** menüpontra navigálva a helyszínek listája válik láthatóvá. A háttérben automatikusan váltakozó tematikus képek (fantasy-stílusú illusztrációk) jelennek meg 30 másodpercenként, amelyek a szerepjátékos hangulatot erősítik.

Minden helyszín egy kártyaként látható, amely tartalmazza:

- A helyszín képét
- A helyszín nevét (nagybetűkkel)
- A pontos címet
- Rövid leírást

A **helyszín kártyájára** kattintva megnyílik a helyszín **részletes nézete**, ahol a teljes leírás olvasható. A **RESERVE** gomb csak bejelentkezett felhasználók számára aktív – kijelentkezve szürkén, letiltva jelenik meg.





3.5.6 Foglalás létrehozása (Reserve)

Egy helyszín részletes nézetében a **RESERVE** gombra kattintva megjelenik a foglalási űrlap. Az űrlap a helyszín adatait (kép, név, cím) a fejlécben mutatja, alatta a kitöltendő mezők és egy naptár widget jelenik meg.

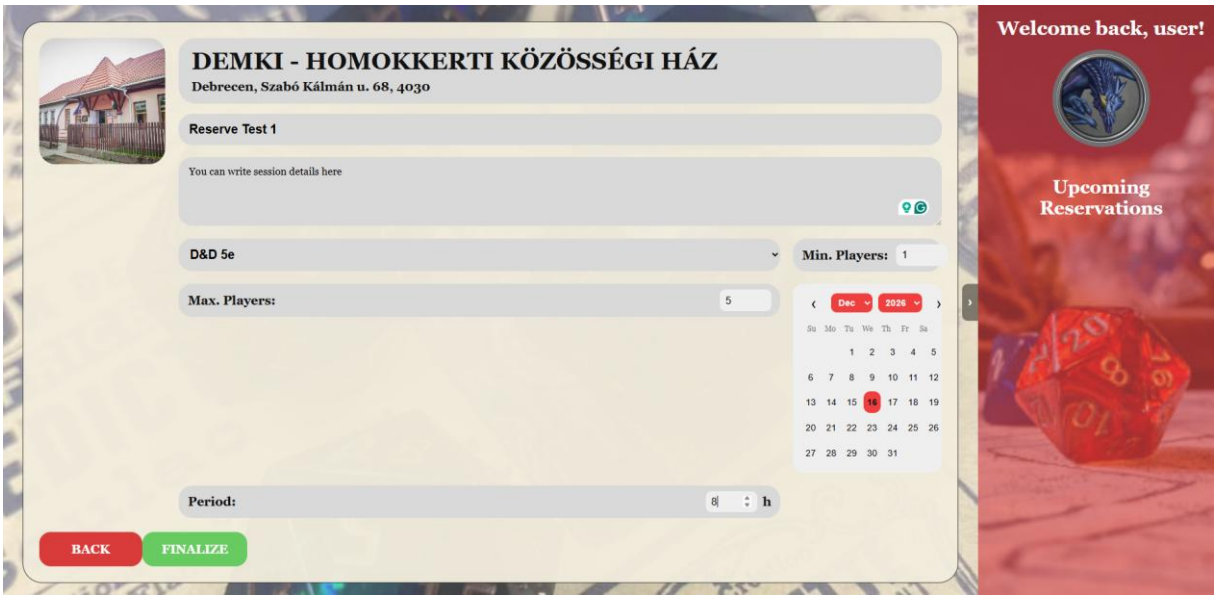
Beviteli mezők:

Mező	Elvárt formátum	Kötelező
Session Title	Szöveges mező. A játékalkalom neve. Max. 255 karakter. Ékezetes karakterek, szóközök, speciális karakterek megengedettek. Nem lehet üres.	Igen
Session description	Többsoros szöveges mező (**textarea**). Szabadon kitölthető leírás. Nincs hosszkorlátozás a felületen. Ékezetes karakterek megengedettek.	Nem
Game System	Legördülő lista (**select**). Választható értékek: „D&D 5e", „Pathfinder 2e", „Call of Cthulhu 7e".	Igen
Min. Players	Szám mező (**type="number"**). 1–12 közötti egész szám. Nem lehet nagyobb a Max. Players értéknél.	Igen
Max. Players	Szám mező (**type="number"**). 1–12 közötti egész szám.	Igen
Naptár	Kattintással választható dátum. A múltbeli napok szürkén jelennek meg és nem választhatók. A már foglalt napok feketén, letiltva jelennek meg. A kiválasztott nap piros háttérrel jelölődik. Hónap- és évválasztó legördülő lista is elérhető.	Igen
Period	Szám mező (órában). 1–12 közötti egész szám. Az esemény időtartama.	Igen

Lehetséges hibaüzenetek:

- *„Please fill in every field before finalizing."* – nem minden kötelező mező van kitöltve (piros szöveg az űrlapon).
- *„Minimum players cannot be greater than maximum players."* – a minimum nagyobb, mint a maximum (piros szöveg az űrlapon).
- „There's a lobby reserved for this date already!" – időpont-ütközés a helyszínen (backend hibaüzenet).
- *„Creating reservation failed."* – általános hiba a létrehozás során (piros sáv a fejlécben).

A foglalás létrehozásakor a lobby **„Pending"** státusszal jön létre. Amikor a csatlakozott játékosok száma eléri a minimum játékosszámot, automatikusan **„Confirmed"** státuszra vált.



3.5.7 Csatlakozás lobbyhoz (Community nézet)

A **Community** fülön az összes elérhető lobby megjelenik kártyás nézetben, 2 oszlopos elrendezésben. Minden lobby kártyán látható:

- A lobby neve
- A Game Master (DM) neve
- A játékrendszer típusa
- A helyszín neve és címe
- A kezdés dátuma és időtartama
- A játékosok létszáma (vizuális sávokkal ábrázolva)
- A státusz (**Confirmed** zölden, **Pending** narancsszínnel)
- A csatlakozott játékosok névsora

A **JOIN PARTY** gomb megnyomásával lehet csatlakozni egy lobbyhoz.

Lehetséges hibaüzenetek:

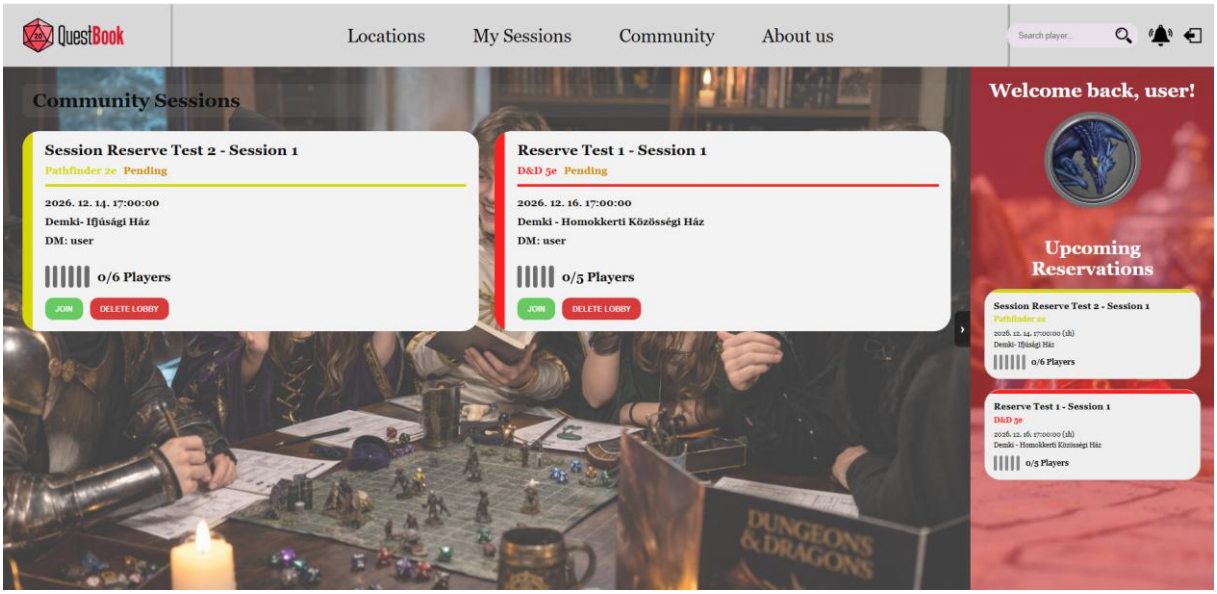
- „You have already joined to this lobby." – dupla csatlakozás.
- „A lobby elérte a maximum játékosszámot." – betelt a lobby.
- „The DM cannot join into his own lobby." – a szervező automatikusan tag.
- „Joining session failed." – általános hiba (piros sáv a fejlécben).

3.5.8 Kilépés lobbyból

A **My Sessions** nézetben vagy a lobby részletes nézetében a **LEAVE PARTY** gombbal lehet kilépni egy lobbyból. Ha a kilépéssel a játékoslétszám a minimum alá csökken, a lobby státusza visszaáll **„Pending"**-re.

3.5.9 Lobby törlése

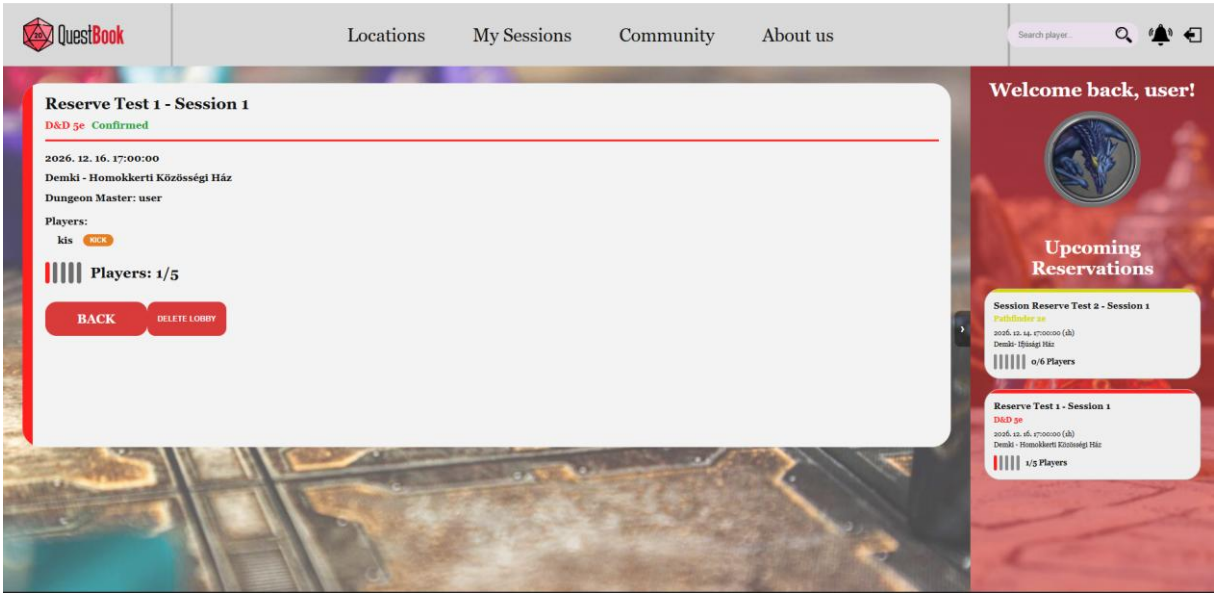
A lobby létrehozója (DM) a **DELETE LOBBY** gombbal törölheti a saját lobbyját.
Admin jogosultságú felhasználó bármely lobbyt törölheti.



Lobby részletes nézet

Egy lobby kártyájára kattintva megnyílik a részletes nézet, ahol megjelenik:

- A lobby teljes neve, játékrendszere és státusza
- A DM neve, helyszín neve és címe
- A kezdés dátuma és időtartama
- A leírás szövege
- A csatlakozott játékosok listája (a DM-en kívül)
- A játékoslétszám vizuális sávokkal
- **LEAVE PARTY / DELETE LOBBY** gombok (jogosultságtól függően)
- DM vagy Admin esetén a játékosok neve mellett **KICK** gomb a játékos eltávolításához



3.5.10 Profil nézet

A jobb oldali sávban a **profilképre** kattintva érhető el a saját profil:

- **Profilkép** – regisztrációkor véletlenszerűen kerül hozzárendelésre (15 különböző illusztráció egyike), de később lecserélhető.
- **Felhasználónév**
- **Reputáció (REP) pontszám** – a közösség visszajelzései alapján számított összeg.
- **Kapott kommentek listája** – más felhasználók által írt megjegyzések, amelyeket a profiloldal tulajdonosa törölhet.

- **Profil szerkesztése:**

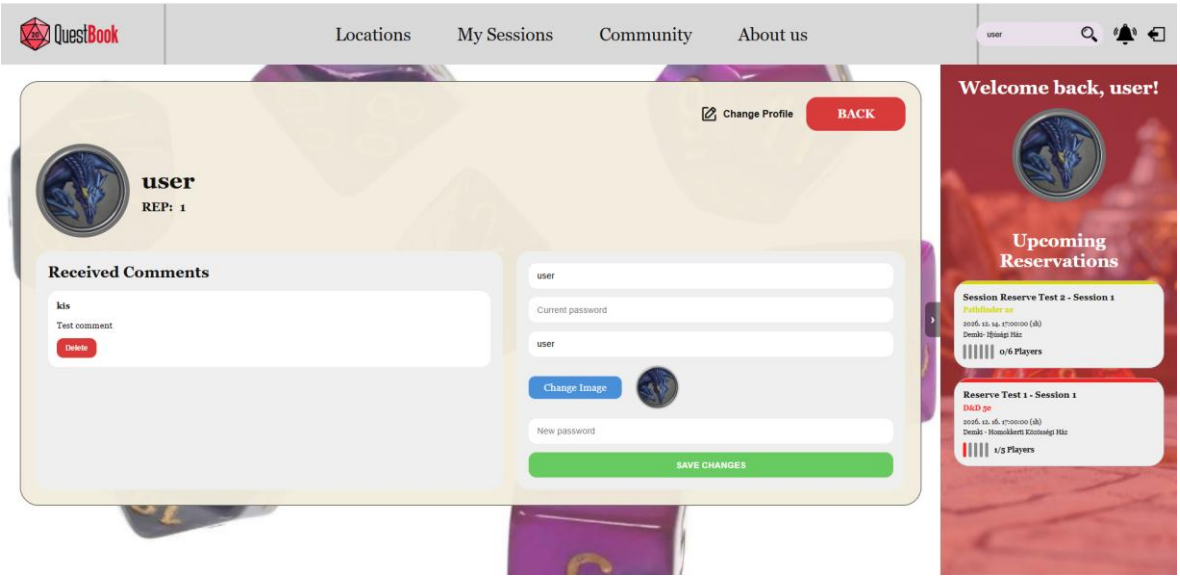
A **Change Profile** gombra kattintva (csak saját profil esetén jelenik meg) megjelenik a szerkesztő panel a kommentek mellett.

Mező	Elvárt formátum	Kötelező
Current name	A jelenlegi felhasználónév (ellenőrzéshez). Pontos egyezés szükséges.	Igen
Current password	A jelenlegi jelszó (ellenőrzéshez). Rejtett megjelenítés. Pontos egyezés szükséges.	Igen
New name	Az új felhasználónév. Nem lehet üres, és nem lehet már foglalt név. Max. 255 karakter. Ékezetes karakterek, szóközök, speciális karakterek megengedettek.	Igen
New image path / url	Új profilkép elérési útvonala vagy URL-je. Szöveges mező. Ha üresen hagyja, a jelenlegi kép megmarad.	Nem
New password	Az új jelszó. Nem lehet üres. Rejtett megjelenítés. Kis- és nagybetűk, számok, speciális karakterek, ékezetes karakterek megengedettek.	Igen

Lehetséges hibaüzenetek:

- „Wrong original password or username" – a jelenlegi név vagy jelszó helytelen.
- „Some fields are empty" – kötelező mező nincs kitöltve.
- „There's a user already with this gmail" – az új név foglalt.
- *„Profile update failed."* – általános hiba (piros sáv a fejlécben).

Sikeres módosítás után az alkalmazás automatikusan frissíti a JWT tokent és a megjelenített adatokat.



3.5.11 Játékos keresése

A navigációs sáv jobb oldalán található **keresőmezőbe** írja be a keresett felhasználó nevét, majd nyomja meg az **Enter** billentyűt vagy a **nagyító ikont**.

A keresőmező jellemzői:

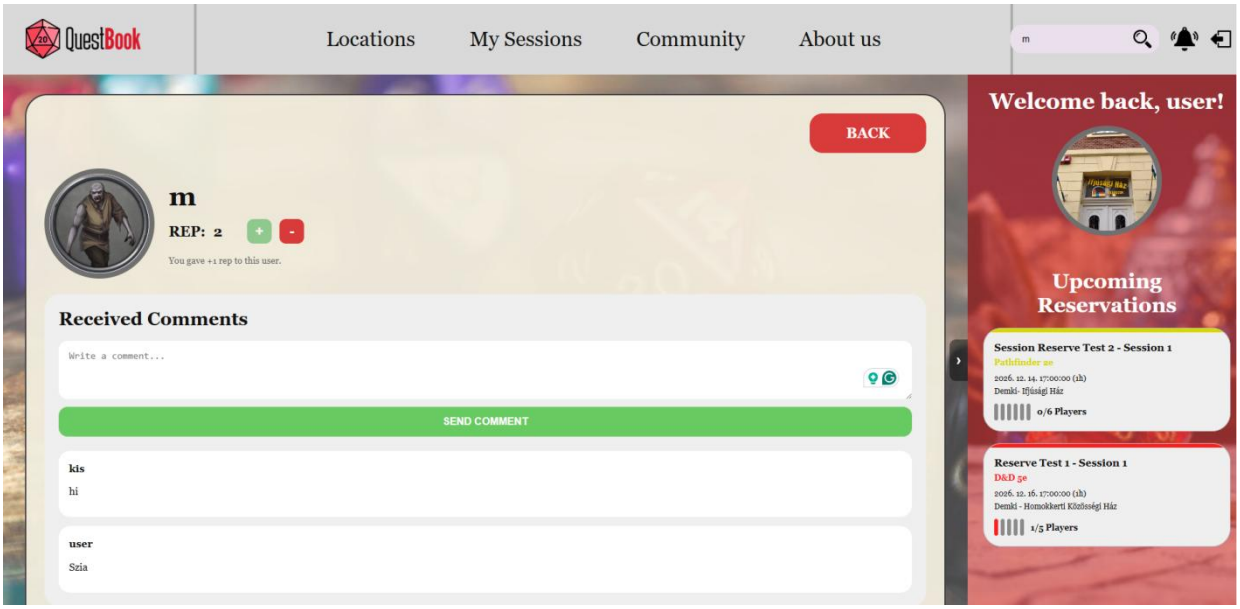
- Szöveges beviteli mező, max. 255 karakter.
- Ékezetes karakterek megengedettek.
- Pontos felhasználónév egyezés szükséges (a keresés nem részleges).

Találat esetén megnyílik a keresett felhasználó profilja, ahol:

- Megtekinthető a neve, profilképe, reputációja és kapott kommentjei.
- **REP szavazatot** adhat (+1 vagy -1). Felhasználónként egyszer szavazhat, de a szavazat iránya módosítható (pl. +1-ről -1-re).
- **Kommentet** írhat a profilra egy többsoros szöveges mezőben (ékezetes karakterek, szóközök, speciális karakterek megengedettek).
- Saját magára nem szavazhat és nem írhat kommentet.

Lehetséges hibaüzenetek:

- *„No player found with that username."* – a keresett felhasználónév nem található (piros sáv a fejlécben).
- „You can't vote on yourself!" – saját profil értékelési kísérlet.
- „You already gave reputation to this user!" – azonos irányú szavazat megismétlése.



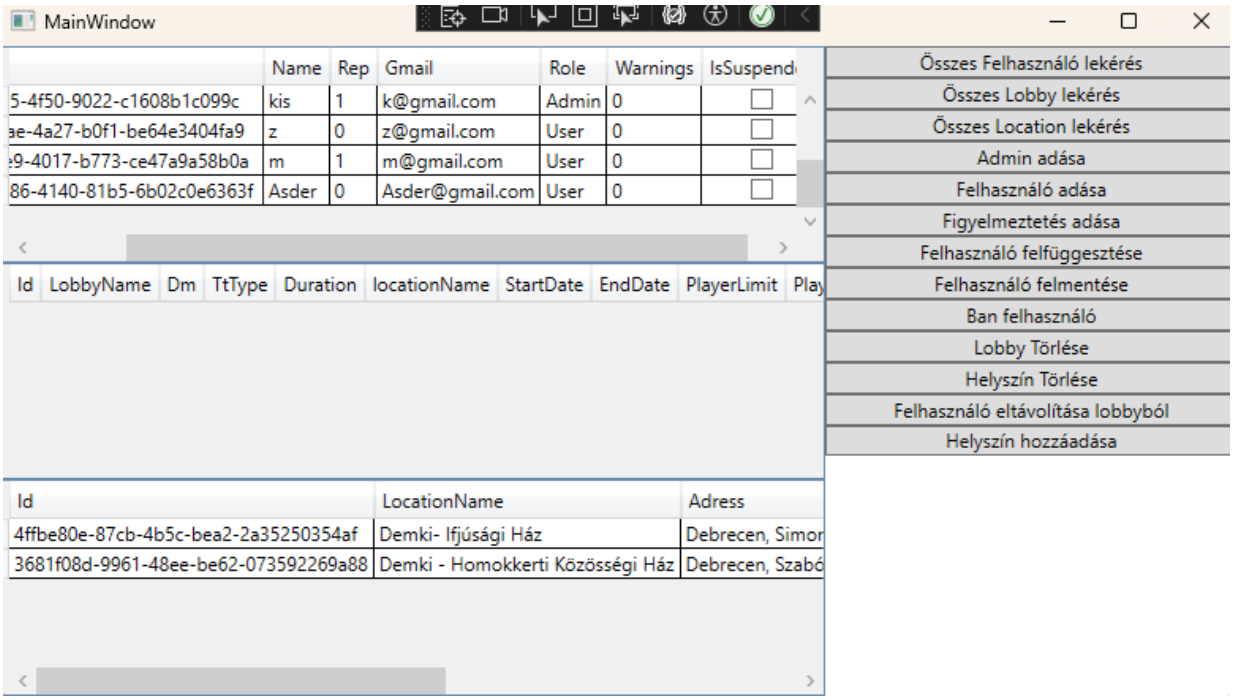
3.5.12 Kijelentkezés

A navigációs sáv jobb szélén lévő **kijelentkezés ikonra** (ajtó ikon) kattintva azonnal kijelentkezhet. A kijelentkezés után:

- A bejelentkezést igénylő funkciók elérhetetlenné válnak (My Sessions, Community menüpontok szürkén jelennek meg).
- A **Locations** nézet jelenik meg.
- A JWT tokenek (access és refresh) törlődnek a böngésző helyi tárolójából.
- A jobb oldali sávban újra megjelennek a **LOGIN** és **REGISTER** gombok.

3.5.13 Admin felület és lehetőségei

Az admin panel egy **önálló WPF asztali alkalmazás**, amely kizárólag **Admin** szerepkörű felhasználók számára érhető el. Az alkalmazás a backend API-val kommunikál HTTP kérésen keresztül.



- **Bejelentkezés az admin panelbe**

Az alkalmazás indításakor egy bejelentkezési ablak jelenik meg három mezővel:

Mező	Elvárt formátum	Kötelező
Felhasználónév	Az admin felhasználónév. Szöveges mező. Pontos egyezés szükséges.	Igen
Gmail	Az admin email cím. Szöveges mező. Érvényes email formátum.	Igen
Jelszó	Az admin jelszó. Rejtett megjelenítés.	Igen

Sikeres bejelentkezés után megnyílik a fő kezelőfelület, amely **három adattáblát** (felhasználók, lobbik, helyszínek) és **műveleti gombokat** tartalmaz.

Hibás bejelentkezési adatok esetén piros szöveges hibaüzenet jelenik meg a bejelentkezési ablakban.

- **Elérhető admin műveletek**

Művelet	**Gomb**	**Leírás**
Felhasználók listázása	**Összes Felhasználó lekérés**	Az összes regisztrált felhasználó megjelenítése (név, email, szerepkör, figyelmeztetések száma, felfüggesztési állapot, reputáció).
Lobbik listázása	**Összes Lobby lekérés**	Az összes lobby megjelenítése (név, DM, típus, helyszín, időpont, létszám, státusz).
Helyszínek listázása	**Összes Location lekérés**	Az összes helyszín megjelenítése (név, cím, leírás, kép).
Admin rang adása	**Admin adása**	A kijelölt felhasználó szerepkörét Admin-ra változtatja.
Felhasználó rang adása	**Felhasználó adása**	A kijelölt admin visszaminősítése User szerepkörre.
Figyelmeztetés	**Figyelmeztetés adása**	A kijelölt felhasználó figyelmeztetés-számlálóját 1-gyel növeli.
Felfüggesztés	**Felhasználó felfüggesztése**	Minimum 3, és 3-mal osztható figyelmeztetés-szám esetén felfüggeszti a felhasználót. A felfüggesztett felhasználó nem tud bejelentkezni a webes felületen.
Felmentés	**Felhasználó felmentése**	A felfüggesztés feloldása, a felhasználó újra bejelentkezhet.
Kitiltás (Ban)	**Ban felhasználó**	Véglegesen törli a felhasználót a rendszerből, és email címét a tiltólistára helyezi. A tiltólistán szereplő email címmel nem lehet újra regisztrálni.
Lobby törlése	**Lobby Törlése**	A kijelölt lobby törlése a rendszerből.
Helyszín törlése	**Helyszín Törlése**	A kijelölt helyszín és a hozzá tartozó lobbik kaszkád törlése.
Játékos eltávolítása	**Felhasználó eltávolítása lobbyból**	A felhasználók és lobbik táblában egyaránt ki kell jelölni egy-egy elemet, majd a kijelölt felhasználó eltávolítható a lobbyból.
Helyszín hozzáadása	**Helyszín hozzáadása**	Felugró ablakban megadható: helyszín neve, cím, leírás, kép URL.

3.5.14 Hibajelzések összefoglalása

A program az alábbi típusú hibajelzéseket jelenítheti meg:

- **Webes felület (frontend)**

Típus	Megjelenés	Példák
Általános hiba	Piros sáv a fejléc alatt (**global-error-banner**)	**Login / register failed., Creating reservation failed., Joining session failed., Leaving session failed., Deleting lobby failed., Profile update failed., Rep update failed., Writing comment failed., Deleting comment failed.**
Validációs hiba	Piros szöveg az űrlapon (foglalási nézetben)	**Please fill in every field before finalizing., Minimum players cannot be greater than maximum players., Passwords do not match.**
Backend válaszüzenetek	Piros sáv vagy szöveg (a szerver válaszából)	**There's an user with this name already, There's a user already with this gmail, This gmail has been banned, Missing username,password or gmail, User not found, Wrong password, This user is suspended!, This gmail has been banned, No token record, Access token expired, There's a lobby reserved for this date already!, You have already joined to this lobby., The DM cannot join into his own lobby., A lobby elérte a maximum játékosszámot., You can't vote on yourself!, You already gave reputation to this user!, User not found, You can only change your profile, Wrong original password or username, Some fields are empty, Profile update failed.**
Keresési hiba	Piros sáv a fejléc alatt	**No player found with that username.**

- **Admin panel (WPF)**

Típus	Megjelenés	Példák
Sikeres művelet	MessageBox felugró ablak	**Warning successfully given: [username]**
Hiba	MessageBox felugró ablak	**Error: 401 - Unauthorized**, `Error:

3.5.15 Elérhetőség

A projekttel kapcsolatos kérdések, visszajelzések és hibajelentések esetén az alábbi csatornákon érhetők el a fejlesztők:

- **GitHub Repository**: https://github.com/SzakacsMate/Vizsgaremek_SzakacsM_BarnaI
  - **Issues**: Hibajelentések és funkciókérések beküldése a GitHub Issues felületén lehetséges.
  - **Discussions**: Általános kérdések és beszélgetések a GitHub Discussions szekcióban.

A projektet **Szakács Máté** és **Barna István** fejlesztették vizsgáremek projektként.

4. Csapatmunka megvalósítása

A projekt tervezését egy Word-dokumentumban kezdtük meg, ahol rögzítettük az alapötleteket és a rendszer fő funkcióit. Ezt a dokumentumot feltöltöttük a GitHub-ra, így minden csapattag számára elérhetővé vált. A verziókezelést és az együttműködést szintén GitHub segítségével oldottuk meg, mivel így könnyen nyomon követhető volt a forráskód aktuális állapota több számítógépről is – ami különösen hasznos volt a folyamatos teremcserék miatt. A GitHub lehetővé tette, hogy párhuzamosan dolgozzunk a projekten anélkül, hogy a módosítások ütköztek volna egymással. Az ütemezés és a feladatok kiosztása a GitHub Projects funkciójával történt, amely átlátható felületet biztosított a teendők követésére és a csapaton belüli koordinációra.

A projekt számunkra komoly kihívást jelentett, mivel összetettebb volt minden korábbi munkánknál. Korábban még nem készítettünk olyan rendszert, amelyben egy weboldalt adatbázissal kellett összekapcsolni, ezért a fejlesztés kezdeti szakasza lassabban haladt. Ekkor elsősorban a weboldal felépítésének és működésének megtervezésére, valamint a feladatok egyeztetésére helyeztük a hangsúlyt.

A csapaton belüli feladatmegosztás a következőképpen alakult:

- **Barna István** felelt a frontend fejlesztéséért, valamint a weboldal alap dizájnjának elkészítéséért Figma segítségével.
- **Szakács Máté** a backend,és admin panel fejlesztését végezte, beleértve az adatbázis megtervezését, a lekérdezések elkészítését, valamint a kapcsolatdiagram (relationship diagram) megrajzolását dbdiagram.net felületen.

A kezdeti funkciók elkészítése után további bővítéseket valósítottunk meg a weboldalon, és Szakács Máté a frontend dizájn továbbfejlesztésében is részt vett.

A csapatmunka eredményeként sikeresen elkészült a tervezett weboldal, amely megfelel az előzetesen kitűzött céloknak.

5. Összegzés

A QuestBook fejlesztése során egy olyan webes platformot hoztunk létre, amely valós igényt elégít ki a társasjátékos és szerepjátékos közösségen belül. A projekt célja egy olyan rendszer megalkotása volt, amely megkönnyíti a játékalkalmak szervezését, a helyszínek és csapatok keresését, valamint a résztvevők értékelését. A fejlesztés során számos technikai és szervezési kihívással találkoztunk, azonban ezek megoldása hozzájárult ahhoz, hogy a végeredmény egy stabilan működő, jól használható alkalmazás legyen.

A rendszer jelenlegi állapotában már alkalmas arra, hogy a felhasználók mindennapi igényeit kiszolgálja, és hatékonyan támogassa a közösségi események szervezését. Ugyanakkor a platform további fejlesztési lehetőségeket is magában hordoz, amelyekkel a jövőben még több funkcióval, kényelmi szolgálatatással bővíthető.
A weboldal elkészítése során elsődleges célunk az volt, hogy egy könnyen kezelhető, felhasználóbarát platformot hozzunk létre. A rendszer angol nyelvű felülete nemzetközi jellegűvé teszi az alkalmazást, és lehetővé teszi, hogy a hazai felhasználókon túl a nemzetközi TTRPG-közösség számára is elérhető legyen.
A felület kialakítása során törekedtünk arra, hogy a funkcionalitáson túl letisztult, esztétikus és egységes dizájn hozzájáruljon a pozitív felhasználói élményhez.

Összességében elmondható, hogy a csapatmunkának és a kitartó fejlesztési folyamatnak köszönhetően sikerült egy olyan alkalmazást létrehoznunk, amely egy valódi piaci rést tölt be, mivel nem találtunk ilyen weboldalt, ami ennyire segítené a csapatok szerveződését, és közösségépítő hatása lesz.

A projekt elkészítése számunkra is rendkívül értékes volt, mivel csapatmunka keretében gyakorolhattuk az eddig tanultakat: frontend és backend fejlesztés, az adatbázistervezés, a verziókezelés. A vizsgaremek elkészítése során olyan komplex feladatokkal találkoztunk, amelyek jól modellezik a későbbi szakmai kihívásokat, így a projekt nemcsak egy működő alkalmazást eredményezett, hanem jelentős gyakorlati tapasztalatot is.